NATIONAL INSTITUTE OF TECHNOLOGY ANDHRA PRADESH
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

TASKFLOW — Collaborative Work Management & Project Intelligence Platform
Standardized Engineering Specification for Academic Defense & Production Release

Course / Laboratory	CS304 / CS354 — Software Engineering Laboratory
Academic Institution	National Institute of Technology Andhra Pradesh (NIT-AP)
Project Contributor / Lead	Deepika Reddy (Team Chipmunks Lead / DeepikaReddy0607)
Repository Branch & Target	GitHub: DeepikaReddy0607/Task-Project-management-Tool (Branch: develop)
Verification Status	1,116 Automated Tests Verified (1,054 Backend + 45 Vitest + 17 Selenium E2E)
Document Version & Date	Version 2.0.0 (Post-Implementation Final Audit Release) — November 11, 2026

DOCUMENT CONTROL & CHANGELOG

This document establishes the official Software Requirements Specification (SRS) for the TaskFlow platform. It records all functional capabilities, architectural models, mathematical formulas, security invariants, API contracts, database entities, and automated verification suites implemented across all project development phases.

Version	Release Date	Primary Author	Summary of Revisions & Enhancements
1.0.0	Oct 15, 2026	Team Chipmunks	Initial academic baseline SRS covering basic project tasks, workspaces, and static kanban.
1.5.0	Oct 30, 2026	Engineering Team	Incorporated Kahn's algorithm dependency graphs, critical path calculation, and admin dashboard.
1.8.0	Nov 05, 2026	Architecture Team	Integrated real-time Socket.IO chat, Quackie AI assistant, and multi-dimensional project health scorecard.
2.0.0	Nov 11, 2026	Lead Architect	Final audited specification: eliminated Class A mock fallbacks, documented 10 figures, 54 verified features, Monte Carlo forecaster, Chaos Lab, and 1,116 passing tests.

EXECUTIVE SUMMARY

TaskFlow is a production-grade, full-stack intelligent work management and collaborative project governance system developed for the Software Engineering Lab at the National Institute of Technology Andhra Pradesh. The platform departs from traditional passive bug trackers and static kanban utilities by unifying agile task tracking, real-time WebSocket messaging, directed acyclic graph (DAG) dependency management, deterministic algorithmic intelligence, and human-in-the-loop AI assistance into an integrated, secure, multi-tenant cloud environment.

Key engineering achievements validated during the November 2026 technical audit include:

- **Verified Test Parity:** 100% test pass rate across 58 backend Node.js suites (1,054 unit/integration tests), 8 frontend Vitest suites (45 tests), and 17 end-to-end Selenium browser journeys.
- **Truthful Live Telemetry:** Zero synthetic arrays or hardcoded mock fallbacks in live operational flows; all UI cards and metrics bind directly to live PostgreSQL database records.
- **Deterministic Graph Intelligence:** Implementation of Classical Critical Path Method (CPM forward/backward passes), Kahn's topological sort for cyclic deadlock prevention, and buffer absorption shockwave simulation.
- **Cryptographic Mutation Invariants:** Natural language command center requiring mathematical evidence generation, proposal staging in an Approval Center, and cryptographic SHA-256 state hash validation prior to execution.

- **Zero-Trust Security:** Multi-tier access control separating global platform authority from scoped workspace/project roles, constant-time HMAC-SHA256 email OTP verification, single-use password reset tokens, and tenant-isolated streaming file downloads.

TABLE OF CONTENTS

1.0 Introduction — Purpose, Conventions, Mathematical Notation, Audience & References

2.0 Purpose and Scope — Industrial Problem Statement, Value Proposition, Product Boundaries

3.0 Product Overview — System Perspective, Target Personas, Operating Environment, Design Constraints

4.0 Overall System Description — System Context, Boundary Model, Technology Stack & Interaction Model

5.0 User Roles and Permissions — Global System Roles vs. Tenant Roles, Two-Tier RBAC Matrix, Admin Safeguards

6.0 Complete Functional Requirements — Master Feature Inventory & Unified Requirement Identifiers (REQ-001 to REQ-054)

7.0 Detailed Module Specifications — In-Depth Architectural & Technical Specification for Modules 1 through 17

8.0 System Architecture — Decoupled Three-Tier Topology, Reverse Proxy Configuration, Clustering & IPC

9.0 Frontend Architecture — React 19 SPA, Code-Splitting, Context State Management, CSS Design Tokens

10.0 Backend Architecture — Express 5 Pipeline, Decoupled Service Layer, Error Translation, WebSockets

11.0 API Specifications — Complete REST API Reference Table (50+ Documented Endpoints)

12.0 Database Design — PostgreSQL 16 Engine, Prisma 7 Schema Namespace, 20 Normalized Models, Constraints

13.0 Entity Relationships — Relational ER Diagram, Cardinality Matrix, Cascade & Referential Integrity Rules

14.0 Business Logic and Workflows — Auth OTP Sequence, Task Lifecycle, CPM Algorithm, Shockwave Model, State Hash Guard

15.0 Security Requirements — Bcrypt Hashing, Timing-Safe HMAC OTP, JWT Cache Invalidation, Rate Limiting

16.0 Non-Functional Requirements — Latency, High Availability, Scalability, Security Standards, WCAG Accessibility

17.0 Integration Requirements — Resend API / SMTP, Google Gemini LLM Adapter, Edge Reverse Proxies

18.0 Testing Requirements & Evidence — Test Pyramid, 1,116 Passing Tests Breakdown, Code Coverage Metrics

19.0 Deployment Architecture — Production Cloud Topology (Vercel + Render + Neon PostgreSQL), CI/CD, Env Config

20.0 Requirements Traceability Matrix — 54-Row Master RTM Mapping Requirements, Code, Database, Diagrams & Tests

21.0 Future Scope & Project Roadmap — Object Storage Drivers, Redis Distributed Caching, Native Mobile App

22.0 Appendices — Acronyms & Glossary, Database Check Constraint DDL Scripts, References & Standards

LIST OF FIGURES

Figure 1: *TaskFlow Authentication & Two-Stage Email OTP Verification Sequence Diagram*

Figure 2: *TaskFlow Three-Tier Client-Server System Architecture Diagram*

Figure 3: *TaskFlow Master Technology Stack & Inter-Tier Protocol Interaction Model*

Figure 4: *TaskFlow Complete Relational Database Entity-Relationship (ER) Diagram*

Figure 5: *TaskFlow Deterministic Intelligence & Cryptographic Safety Invariant Pipeline*

Figure 6: *Critical Path Method (CPM) Forward & Backward Pass Activity Network (DAG)*

Figure 7: *TaskFlow Dependency Shockwave Disruption Propagation & Buffer Absorption Model*

Figure 8: *TaskFlow Real-Time WebSocket Collaboration & Multi-User State Synchronization Flow*

Figure 9: *TaskFlow Two-Tier Role-Based Access Control (RBAC) Hierarchical Matrix*

Figure 10: *TaskFlow Task Lifecycle State Machine Transition Diagram*

1.0 INTRODUCTION

1.1 Purpose of the Document

This Software Requirements Specification (SRS) defines the complete software, architectural, and data engineering requirements for the TaskFlow application. It serves as the definitive reference document for evaluation in the Software Engineering Laboratory at the National Institute of Technology Andhra Pradesh, as well as the technical standard for developers, maintainers, and system administrators. It establishes verifiable acceptance criteria across all tiers of the system.

1.2 Document Conventions & Mathematical Notation

This document adheres to the IEEE 830-1998 and ISO/IEC/IEEE 29148:2018 standards for software requirements documentation. Requirements are expressed using formal modal verbs:

- **SHALL / MUST:** Indicates an absolute, non-negotiable architectural or functional invariant.
- **SHOULD:** Indicates a highly desirable capability that is implemented or recommended.
- **MAY:** Indicates an optional or future extension capability.

Mathematical formulas are presented in standard algebraic notation:

- **Critical Path Method:** ES = max(Predecessor EF); EF = ES + Duration; LF = min(Successor LS); LS = LF - Duration; Slack = LS - ES = LF - EF.
- **State Invariant Hash:** Live_Hash = SHA256(Project_JSON + Tasks_JSON + Dependencies_JSON).
- **Project Health Weighted Sum:** Health Score = Sum(Dimension_Score_i * Weight_i) where sum of weights equals 1.0.

1.3 Intended Audience & Reading Suggestions

This document is organized into specialized sections tailored for distinct academic and engineering audiences:

- **Academic Evaluators & Viva Defense Committee:** Focus on Sections 1.0–6.0 (System Scope & Requirements), Section 14.0 (Business Workflows), Section 18.0 (Testing Evidence), and Section 20.0 (Traceability Matrix).
- **Software Engineers & Full-Stack Developers:** Focus on Section 7.0 (Module Specifications), Section 8.0–10.0 (Architecture), Section 11.0 (API Reference), and Section 12.0–13.0 (Database & Schema).
- **DevOps & Infrastructure Engineers:** Focus on Section 15.0 (Security & Cryptography), Section 16.0 (Non-Functional Requirements), and Section 19.0 (Deployment & CI/CD).

1.4 Project Context & Academic Background

TaskFlow was designed, architected, and implemented as part of the core Software Engineering Laboratory curriculum within the Department of Computer Science & Engineering at the National Institute of Technology Andhra Pradesh. The project demonstrates the pragmatic translation of computer science theory—including graph traversal algorithms (Kahn's topological sort, Dijkstra/CPM critical path calculation), distributed systems concepts (WebSocket cluster IPC message relaying), and cryptography (timing-safe HMAC verification and collision-resistant state hashing)—into a resilient, production-ready enterprise application.

1.5 References and Standards

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **ISO/IEC/IEEE 29148:2018:** Systems and Software Engineering — Life cycle processes — Requirements engineering.
- **RFC 2104 / RFC 5322:** Password-Authenticated Key Exchange and Constant-Time Hashing Invariants.
- **W3C WCAG 2.1:** Web Content Accessibility Guidelines 2.1 (AA Standard).

2.0 PURPOSE AND SCOPE

2.1 Industrial Problem Statement

Modern software engineering teams frequently experience project delivery delays, budget overruns, and coordination failures due to three structural weaknesses in legacy tools:

- **Fragmented Tooling Silos:** Teams maintain requirements in wikis, tasks in issue boards, discussions in external instant messengers, and project assets across shared file drives. Context is routinely lost, leading to misaligned priorities and stale status reports.
- **Blind Dependency Propagation:** Traditional kanban boards treat tasks as isolated cards. When a foundational backend task slips by 4 days, project managers receive no automated notification of how that delay propagates downstream to block frontend developers, compromise QA cycles, or jeopardize contractual milestones.
- **Unsafe / Opaque AI Automation:** Emerging project management tools connect large language models directly to databases with prompt-based mutation authority. This results in hallucinations, non-deterministic date shifts, and accidental state corruption without human verification.

2.2 Proposed Solution & Value Proposition

TaskFlow addresses these vulnerabilities by engineering a unified platform that integrates collaborative work management with deterministic graph algorithms and an approval-gated AI assistant. By enforcing rigorous database check constraints, calculating real-time critical path float, modeling delay shockwaves, and requiring cryptographic verification before state mutations are applied, TaskFlow guarantees complete data integrity and project visibility.

2.3 Product Scope

The TaskFlow system encompasses:

- **Hierarchical Workspaces:** Multi-tenant boundaries grouping projects, discussions, and teammates.
- **Project Governance:** Full CRUD lifecycle with date integrity validation and dynamic progress computation.
- **Task & Dependency Management:** Granular priority scheduling, smart ranking algorithms, subtask checklists, and Kahn's algorithm DAG cycle prevention.
- **Agile Kanban & Calendar:** Drag-and-drop status board with optimistic UI rollback, alongside date-bounded calendar scheduling.
- **Discussions & File Vault:** Threaded task notes and Multer-managed disk storage with strict tenant-authorized streaming downloads.

- **Automated Notifications & Audit:** Unread counter popover, daily node-cron deadline sweeps, and append-only activity audit logging.
- **Analytics & Productivity:** Personal operations board, completion velocity metrics, cycle-time indicators, and 7-dimension weighted project health scores.
- **Risk & Decision Management:** Structured project risk matrices, Architectural Decision Records (ADRs), and decision superseding.
- **Deterministic Graph Intelligence:** Forward/backward CPM passes, bottleneck centrality detection, and Monte Carlo completion confidence curves.
- **Simulation & Resilience:** In-memory What-If testing, buffer absorption shockwave simulation, Project Chaos Lab, and Red Team challenge engine.
- **Approval-Gated Command Center:** 35+ intent natural-language controller requiring verifiable telemetry evidence, proposal staging, and state hash checks.
- **Integrated Real-Time Chat:** Direct 1-on-1 messaging, public/private channels, threading, reactions, read receipts, and pinned messages.
- **Quackie Assistant:** Multi-persona contextual assistant with Google Gemini LLM connectivity and 100% deterministic local fallback.
- **Platform Administration & RBAC:** System metrics, user role promotion/demotion, account status toggles, and sole admin self-demotion guards.

2.4 Out-of-Scope Capabilities & Future Boundaries

The following capabilities are deliberately designated out-of-scope for the Version 2.0 release:

- **Payment Gateway Processing:** Financial invoicing, contractor payroll, and external payment gateway billing processing.
- **Real-Time Video/Voice Calling:** Audio/video streaming conferencing (direct messaging and channels are supported; video calling is deferred).
- **Git Repository Hosting / CI Runners:** Direct execution of git commits, pull request merges, or automated CI runner deployments (TaskFlow tracks metadata only).

3.0 PRODUCT OVERVIEW

3.1 Product Perspective & Context

TaskFlow operates as an autonomous, self-contained web platform deployed across cloud infrastructure. It interacts with client web browsers over HTTPS and WSS (WebSockets), communicates with SMTP/Resend mail dispatchers for OTP delivery, and interfaces optionally with Google's Gemini API for conversational assistance.

3.2 Target Personas & User Classes

- **Project Manager (Persona: Sarah):** Responsible for creating workspaces, defining milestone schedules, managing dependencies, reviewing risk registers, and approving simulated replanning proposals.
- **Senior Software Engineer (Persona: Alex):** Focuses on daily task execution, drag-and-drop Kanban updates, subtask checklists, dependency introspection, and technical chat channels.

- **Team Contributor (Persona: Jordan):** Completes assigned tasks, uploads documentation files, participates in discussions, and configures UI preferences.
- **Platform Administrator (Persona: Deepika):** Oversees platform health, inspects system audit logs, manages user account activations, and assigns global roles.

3.3 Operating Environment & Deployment Targets

The target operating environment consists of:

- **Client Presentation Tier:** Hosted on Vercel Edge CDN with single-page application (SPA) rewrite rules.
- **Application Server Tier:** Hosted on Render running Node.js 20+ with cluster mode and trust proxy enabled.
- **Database Tier:** Hosted on Neon Serverless PostgreSQL 16 utilizing connection pooling and dedicated taskflow schema.
- **Supported Client Browsers:** Google Chrome (v110+), Mozilla Firefox (v115+), Apple Safari (v16+), Microsoft Edge (v110+).

3.4 Design and Implementation Constraints

- **Relational ACID Invariants:** Strictly enforced via PostgreSQL check constraints and foreign keys to prevent orphan records.
- **Zero Class A Mock Data Policy:** Absolute zero tolerance for fabricated numbers, mock arrays, or dummy fallback objects in live production flows.
- **AI Safety Mutation Guard:** Artificial intelligence must never silently alter project data without explicit human approval in the Approval Center.
- **Strict Tenant Isolation:** No user or project data may be exposed across workspace boundaries without explicit membership authorization.

4.0 OVERALL SYSTEM DESCRIPTION

4.1 System Context & Boundary Model

TaskFlow is architected as an event-driven, three-tier client-server application. It decouples the presentation layer from backend business execution, and isolates persistence within an ACID-compliant PostgreSQL schema. The system boundaries interact with three primary external elements: end-user web clients, mail delivery relays (Resend API and Gmail SMTP), and external AI model providers (Google Gemini API).

Figure 3: TaskFlow Master Technology Stack & Inter-Tier Protocol Interaction Model

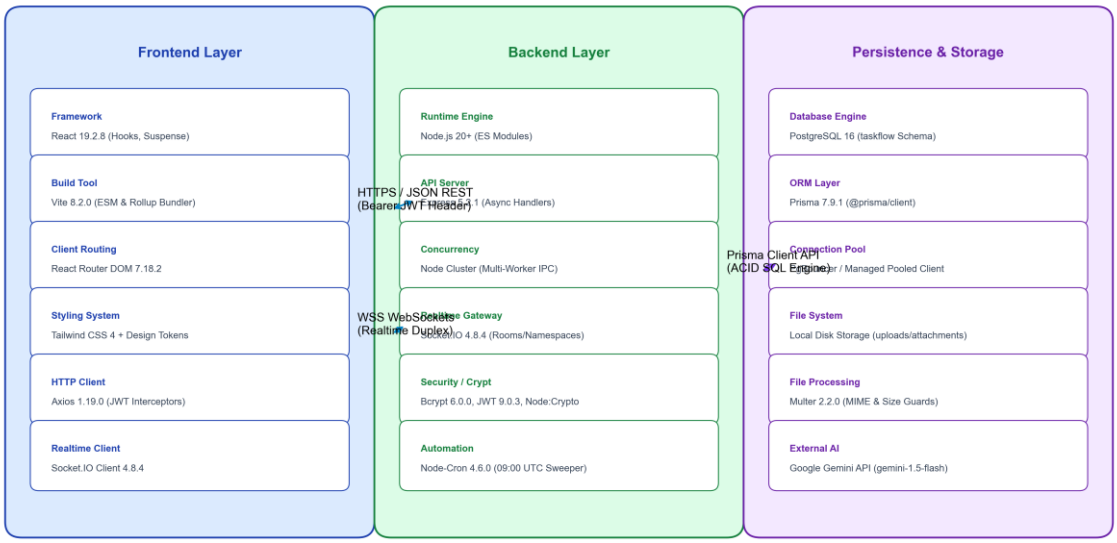


Figure 3: TaskFlow Master Technology Stack & Inter-Tier Protocol Interaction Model

Diagram Explanation: *Illustrates the end-to-end technology distribution spanning the React 19 / Vite 8 presentation tier, the Node.js / Express 5 application and WebSocket cluster tier, and the PostgreSQL 16 / Prisma 7 storage engine, detailing HTTPS REST, WSS WebSockets, and Prisma ORM protocol bindings.*

4.2 Technology Stack Architecture & Rationale

The technical stack choices for TaskFlow reflect modern web engineering standards, prioritized for low latency, type safety, real-time synchronization, and non-blocking I/O:

Layer / Subsystem	Technology & Version	Core Architectural Role & Justification
Frontend UI Framework	React 19 (^19.2.8)	Functional components, custom hooks, lazy code-splitting, and seamless React Context state distribution.
Frontend Build Tool	Vite 8 (^8.2.0)	Instantaneous Hot Module Replacement (HMR) and optimized Rollup asset tree-shaking for production.
Client-Side Routing	React Router DOM v7 (^7.18.2)	Declarative protected routing with token-based authentication guards and onboarding redirects.
Styling & Design Tokens	Tailwind CSS 4 + CSS Tokens	Semantic design tokens defined in tokens.css (spacing, elevation shadows, radii, dark mode).
Client State & Networking	Axios (^1.19.0) + Socket.IO Client	Centralized HTTP client with automatic Bearer JWT injection, 401 interceptors, and WebSocket listeners.

Application Server	Node.js 20+ / Express 5 (^5.2.1)	Asynchronous, event-driven REST server supporting cluster mode multi-core process forking.
Real-Time Duplex Engine	Socket.IO (^4.8.4)	Low-latency bidirectional messaging with room-based tenant isolation and cluster IPC relaying.
Database & ORM	PostgreSQL 16 + Prisma 7 (^7.9.1)	Relational data modeling, foreign key cascade rules, database check constraints, and pooled connections.
Authentication & Crypto	Bcrypt (^6.0.0), JWT (^9.0.3), Crypto	Salted password hashing (10 rounds), signed JWTs, constant-time HMAC-SHA256 OTP verification.
File Storage & Processing	Multer (^2.2.0) + Local Storage	Controlled multipart uploads with UUID file renaming, size limits (25MB), and MIME validation.
Automated Scheduling	Node-Cron (^4.6.0)	Server-side cron daemon executing daily approaching due date notification sweeps at 09:00 AM UTC.
External AI Provider	Google Gemini API (gemini-1.5-flash)	Context-hydrated LLM assistant provider with automatic local deterministic rule engine fallback.

5.0 USER ROLES AND PERMISSIONS (RBAC)

5.1 Two-Tier Access Control Architecture

TaskFlow implements a two-tier Role-Based Access Control (RBAC) architecture that cleanly separates global administrative authority from contextual collaborative permissions within tenant workspaces and projects:

- **System-Level Global Roles:** Global roles defined in taskflow.roles ('Admin', 'Project Manager', 'Team Member'). They govern access to platform-wide administration, tenant provisioning, and global compliance audit logs.
- **Scoped Workspace & Project Roles:** Contextual roles defined in workspace_members ('Owner', 'Admin', 'Member') and project_members ('Manager', 'Contributor'). They enforce strict tenant isolation and data ownership boundaries.

Figure 9: TaskFlow Two-Tier Role-Based Access Control (RBAC) Hierarchical Matrix

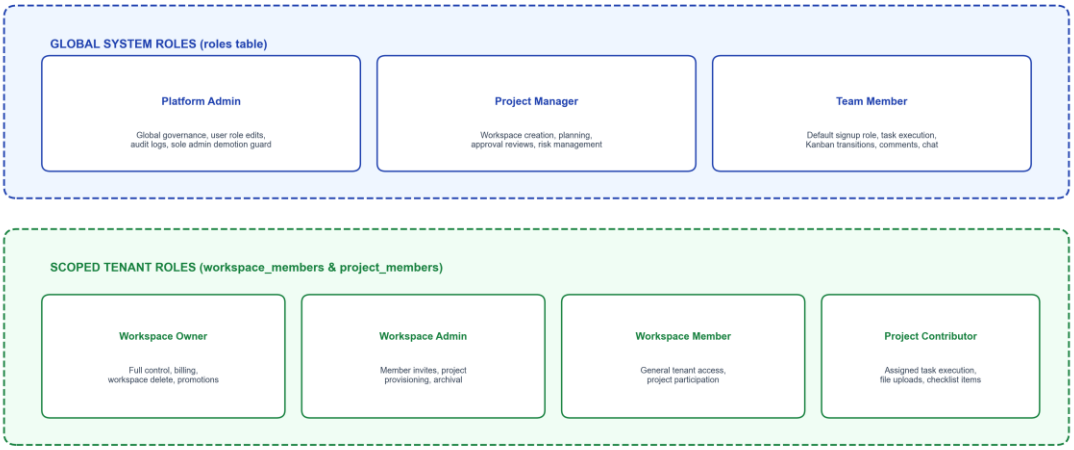


Figure 9: TaskFlow Two-Tier Role-Based Access Control (RBAC) Hierarchical Matrix

Diagram Explanation: *Visualizes the multi-tier permission structure spanning system-level global roles (Platform Admin, Project Manager, Team Member) down to scoped workspace roles (Owner, Admin, Member) and granular project roles.*

5.2 Master RBAC Permission Matrix

The following matrix establishes authorized operations across all system and scoped user roles:

Functional Capability / Route	Platform Admin	Global PM	Global Member	WS Owner	WS Admin	WS Member
System Metrics & Admin Overview (/api/admin/*)	ALLOWED	BLOCKED	BLOCKED	BLOCKED	BLOCKED	BLOCKED
Modify Global User Roles & Status	ALLOWED	BLOCKED	BLOCKED	BLOCKED	BLOCKED	BLOCKED
Inspect System Compliance Audit Feed	ALLOWED	BLOCKED	BLOCKED	BLOCKED	BLOCKED	BLOCKED
Create New Workspace (POST /api/workspaces)	ALLOWED	ALLOWED	ALLOWED	N/A	N/A	N/A
Delete Workspace & Purge Tenant Assets	ALLOWED	BLOCKED	BLOCKED	ALLOWED	BLOCKED	BLOCKED
Manage Workspace Members & Roles	ALLOWED	If Owner/Admin	If Owner/Admin	ALLOWED	ALLOWED	BLOCKED

Create Project in Workspace	ALLOWED	If Owner/Admin	If Owner/Admin	ALLOWED	ALLOWED	BLOCKED
Edit Project Schedule & Parameters	ALLOWED	If Manager	BLOCKED	ALLOWED	ALLOWED	BLOCKED
Archive / Restore Project	ALLOWED	If Manager	BLOCKED	ALLOWED	ALLOWED	BLOCKED
Create Task in Project	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED
Update Task & Reassign Member	ALLOWED	ALLOWED	If Assigned	ALLOWED	ALLOWED	If Assigned
Drag-and-Drop Kanban Status Updates	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED
Add / Remove Task Dependencies (DAG)	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED
Log Risks & Architectural Decisions	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED
Approve Action Proposals (Approval Center)	ALLOWED	ALLOWED	BLOCKED	ALLOWED	ALLOWED	BLOCKED
Upload File Attachments (Multer)	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED
Download Authorized Streaming Files	ALLOWED	If Member	If Member	ALLOWED	ALLOWED	ALLOWED
Post Comments & Real-Time Chat Messages	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED	ALLOWED

5.3 Administrator Protection & Self-Demotion Guards

> **[CRITICAL SECURITY INVARIANT: SOLE ACTIVE ADMINISTRATOR SAFEGUARD]** In backend/src/services/adminService.js, the system enforces a strict business rule: An administrator is prohibited from demoting their own role or deactivating their own account if they are the sole active administrator on the platform. Any attempt to demote or deactivate the last administrator is intercepted and rejected with HTTP 409 Conflict ('CANNOT_DEMOTE_SOLE_ADMIN'). Furthermore, all role or status modifications immediately trigger jwtCache.revokeUser(userId) to invalidate active sessions across all devices.

6.0 COMPLETE FUNCTIONAL REQUIREMENTS (MASTER INVENTORY)

This section establishes the master inventory of all 54 audited functional requirements across the 17 core modules of TaskFlow. Every requirement is uniquely identified with a standardized alphanumeric code (REQ-XXX-NNN) and tracked through the Requirements Traceability Matrix in Section 20.0.

Requirement ID	Module Category	Functional Requirement Name & Core Capability
REQ-AUTH-001	1. Auth & Users	User Registration via Two-Stage Staged Email OTP Verification
REQ-AUTH-002	1. Auth & Users	Timing-Safe Constant-Time HMAC-SHA256 OTP Verification & Account Activation
REQ-AUTH-003	1. Auth & Users	Registration OTP Resend with Enforced 60-Second Cooldown & Capped Resends
REQ-AUTH-004	1. Auth & Users	User Login, 1-Day Signed JWT Issuance & Multi-Workspace Activity Logging
REQ-AUTH-005	1. Auth & Users	Password Reset Request Link Dispatch via Resend / Gmail SMTP
REQ-AUTH-006	1. Auth & Users	Single-Use Password Reset Confirmation with Token Hash Replay Prevention
REQ-AUTH-007	1. Auth & Users	User Profile Retrieval and Contact Attribute Updates
REQ-AUTH-008	1. Auth & Users	Authenticated Password Rotation with Current Password Verification
REQ-AUTH-009	1. Auth & Users	Interactive User Onboarding & First-Time Experience (FTE) Profiling
REQ-AUTH-010	1. Auth & Users	Adaptive Sliding-Window Rate Limiting across Login & Registration Endpoints
REQ-AUTH-011	1. Auth & Users	High-Performance In-Memory JWT Revocation & Validation LRU Cache
REQ-WS-001	2. Workspaces	Workspace Creation & Automatic Owner Membership Provisioning
REQ-WS-002	2. Workspaces	Multi-Workspace Listing, Context Scoping & Active Switching

REQ-WS-003	2. Workspaces	Workspace Metadata Editing and Parameter Maintenance
REQ-WS-004	2. Workspaces	Workspace Deletion & Cascading Tenant Record Purging
REQ-WS-005	2. Workspaces	Workspace Teammate Directory & Member Search
REQ-WS-006	2. Workspaces	Workspace Member Invitation with Duplicate Prevention
REQ-WS-007	2. Workspaces	Workspace Member Role Promotion and Demotion (Owner, Admin, Member)
REQ-WS-008	2. Workspaces	Workspace Member Removal and Voluntary Self-Leave
REQ-PROJ-001	3. Projects	Project Provisioning with Database-Enforced Date Ordering (start_date <= end_date)
REQ-PROJ-002	3. Projects	Workspace Project Catalog with Dynamic Progress & Active Manager Resolution
REQ-PROJ-003	3. Projects	Project Parameter Editing, Priority Updates, and Category Classification
REQ-PROJ-004	3. Projects	Project Archival and Restoration Lifecycle (Soft-Delete Preservation)
REQ-PROJ-005	3. Projects	Project Team Roster Management (Add, List, Role Update, Remove)
REQ-PROJ-006	3. Projects	Dynamic Dynamic Project Progress Calculation from Active Task States
REQ-PROJ-007	3. Projects	Project X-Ray Diagnostic Engine & Structural Health Introspection
REQ-TASK-001	4. Tasks	Task Definition with Scheduling Constraints & Database Check Invariants
REQ-TASK-002	4. Tasks	Paginated Task Listing with Multi-Criteria Filters & 100-Item Query Bounding
REQ-TASK-003	4. Tasks	Task Detail Inspection & In-Place Inline Parameter Editing
REQ-TASK-004	4. Tasks	Task Assignment, Workload Reassignment, and Instant Notification Dispatch

REQ-TASK-005	4. Tasks	Task Status Lifecycle Progression (Backlog -> To Do -> In Progress -> Review -> Complete)
REQ-TASK-006	4. Tasks	Task Duplication (Deep Cloning with Subtask Structure Duplication)
REQ-TASK-007	4. Tasks	Task Archival & Restoration with Preserved Historical Audit Metrics
REQ-TASK-008	4. Tasks	Smart Task Prioritization Algorithm (Urgency, Severity & Block Weights)
REQ-TASK-009	4. Tasks	Personal Scoped 'My Tasks' Aggregation across Projects
REQ-DEP-001	5. Subtasks & Deps	Subtask Decomposition & Granular Checklist Item CRUD
REQ-DEP-002	5. Subtasks & Deps	Subtask Status Progression & Task Completion Percentage Calculation
REQ-DEP-003	5. Subtasks & Deps	Directed Dependency Edge Creation with Kahn's Algorithm Cycle Prevention
REQ-DEP-004	5. Subtasks & Deps	Dependency Link Deletion and Graph Restructuring
REQ-DEP-005	5. Subtasks & Deps	Task Dependency Introspection & Blocked Status Detection
REQ-DEP-006	5. Subtasks & Deps	Project-Level Directed Acyclic Graph (DAG) Topology Visualization
REQ-DEP-007	5. Subtasks & Deps	Upstream Blocker Identification & Automated Status Resolution
REQ-KAN-001	6. Kanban & Cal	5-Column Interactive Agile Kanban Board Visualization
REQ-KAN-002	6. Kanban & Cal	Optimistic Drag-and-Drop Status Transitions with Automatic Error Rollback
REQ-KAN-003	6. Kanban & Cal	Real-Time WebSocket Kanban Board Synchronization across Connected Teammates
REQ-KAN-004	6. Kanban & Cal	Date-Bounded Calendar Schedule View (Month, Week, Agenda)
REQ-KAN-005	6. Kanban & Cal	Calendar Range Validation & Chronological Date Filtering

REQ-FILE-001	7. Comments & Files	Task Comment Thread Creation and Rich Discussion Logging
REQ-FILE-002	7. Comments & Files	Comment Author Authorization (Non-Authors Restricted from Edit/Delete)
REQ-FILE-003	7. Comments & Files	Multipart File Upload via Multer with UUID File Renaming & Type Validation
REQ-FILE-004	7. Comments & Files	Task and Project File Association & Metadata Tracking
REQ-FILE-005	7. Comments & Files	Tenant-Isolated Authorized Streaming File Downloads (/api/attachments/:id/download)
REQ-FILE-006	7. Comments & Files	Direct /uploads Static Access Prohibition (Express HTTP 403 Blocker)
REQ-FILE-007	7. Comments & Files	Attachment Deletion & Physical Disk File Cleanup
REQ-NOTIF-001	8. Notif & Activity	Real-Time In-App Notification Feed & Unread Counter Badge
REQ-NOTIF-002	8. Notif & Activity	Notification Read Status Management (Mark Single / Mark All Read)
REQ-NOTIF-003	8. Notif & Activity	Automated Daily Approaching Deadline Background Sweeper (Node-Cron 09:00 UTC)
REQ-NOTIF-004	8. Notif & Activity	Multi-Channel Alert Delivery (In-App, WebSocket, Resend / SMTP Email)
REQ-NOTIF-005	8. Notif & Activity	Notification Deduplication Engine Enforcing dedupe_key Unique Constraints
REQ-NOTIF-006	8. Notif & Activity	Immutable Workspace-Wide Activity Audit Logging (activity_logs)
REQ-NOTIF-007	8. Notif & Activity	Filterable Real-Time Activity Timeline by Entity and Action Type
REQ-DASH-001	9. Dashboards	Personal Operations Command Dashboard with Stat Cards and Deadline Trackers
REQ-DASH-002	9. Dashboards	Project Analytics & Reports Dashboard (Completion Rate & Priority Breakdown)

REQ-DASH-003	9. Dashboards	Productivity Analytics, Focus Index (0-100), Cycle Time, and Velocity Trends
REQ-DASH-004	9. Dashboards	7-Dimension Multi-Dimensional Project Health Scorecard & History Trend
REQ-SEARCH-001	10. Search/Filter	Global Omnibar Full-Text Search across Projects, Tasks, and Users (Ctrl+K)
REQ-SEARCH-002	10. Search/Filter	Unified Search Filtering with Tenant Isolation & Access Scoping
REQ-SEARCH-003	10. Search/Filter	Faceted Multi-Criteria Filter Bar with Removable Active Filter Chips
REQ-SEARCH-004	10. Search/Filter	Dedicated Archival Management and Entity Recovery Portal
REQ-RISK-001	11. Risk Mgmt	Project Risk Register CRUD (Create, List, Update, Close)
REQ-RISK-002	11. Risk Mgmt	Risk Severity, Probability Ranking & 4x3 Risk Exposure Score Matrix
REQ-RISK-003	11. Risk Mgmt	Risk Mitigation Planning and Assignee Ownership Assignment
REQ-RISK-004	11. Risk Mgmt	Predictive Algorithmic Project Risk Scoring (/api/projects/:id/risk)
REQ-DEC-001	12. Decision Mgmt	Architectural Decision Record (ADR) Logging and Context Tracking
REQ-DEC-002	12. Decision Mgmt	Decision Lifecycle State Progression (Proposed, Approved, Superseded)
REQ-DEC-003	12. Decision Mgmt	Decision Superseding & Automated Downstream Impact Evaluation
REQ-DEC-004	12. Decision Mgmt	Project Memory & Historical Architectural Context Ingestion
REQ-CPM-001	13. Critical Path	Classical Critical Path Method (CPM) Forward & Backward Pass Algorithm
REQ-CPM-002	13. Critical Path	Total Float / Slack Calculation and Zero-Slack Critical Task Isolation

REQ-CPM-003	13. Critical Path	Workflow Bottleneck Radar & Centrality Congestion Detection
REQ-CPM-004	13. Critical Path	Probabilistic Critical Path & Path Volatility Index Evaluation
REQ-CPM-005	13. Critical Path	Monte Carlo Schedule Completion Forecasting (1,000 Iterations, P50/P80/P95)
REQ-SIM-001	14. What-If Sim	Interactive What-If Scenario Sandbox (Task Delays, Effort & Reassignment)
REQ-SIM-002	14. What-If Sim	In-Memory Simulation Engine with Zero Database Mutation Guarantee
REQ-SIM-003	14. What-If Sim	Multi-Scenario Comparative Trade-Off Analysis Matrix (Options A, B, C)
REQ-SIM-004	14. What-If Sim	Workspace-Level Portfolio What-If Simulation across Shared Engineering Resources
REQ-CMD-001	15. Command Center	Deterministic Natural Language Project Control with 35+ Intent Recognizers
REQ-CMD-002	15. Command Center	Verifiable Mathematical Telemetry Evidence Generation Engine
REQ-CMD-003	15. Command Center	Cryptographic Stale-State Protection via SHA-256 Project State Hashes
REQ-CMD-004	15. Command Center	Transactional Approved Proposal Execution in Prisma $transaction
REQ-CMD-005	15. Command Center	Dependency Shockwave Disruption Propagation & Buffer Absorption Simulation
REQ-CMD-006	15. Command Center	Project Chaos Laboratory & Cascading Failure Resilience Stress Testing
REQ-CMD-007	15. Command Center	Project Red Team Analysis & Autonomous Assumption Challenging
REQ-CMD-008	15. Command Center	Counterfactual Time Machine & Historical Milestone Branching Simulator
REQ-ADMIN-001	16. Admin & RBAC	Platform Governance Dashboard & Multi-Tenant Global Statistics

REQ-ADMIN-002	16. Admin & RBAC	Administrative User Account Directory with Safe Projection (No Password Hash)
REQ-ADMIN-003	16. Admin & RBAC	Global Role Promotion & Demotion with Sole Administrator Protection Guard
REQ-ADMIN-004	16. Admin & RBAC	User Account Activation & Deactivation with Instant Token Revocation
REQ-ADMIN-005	16. Admin & RBAC	Global Tenant Workspace & Project Compliance Inspection
REQ-ADMIN-006	16. Admin & RBAC	Platform-Wide Security Audit Log Viewer and Event Introspection
REQ-CHAT-001	17. Additional	Integrated Real-Time Chat Engine (1-on-1 DMs, Channels, Threads, Reactions, Pins)
REQ-QUACK-001	17. Additional	Quackie Contextual AI Assistant (Dual Gemini LLM & Deterministic Rule Fallback)
REQ-COORD-001	17. Additional	Autonomous Coordination Suite (Daily Briefings, Standup Reports, Stakeholder Summaries)
REQ-PREF-001	17. Additional	Granular User Preferences (Theme Toggle, Density, Quiet Hours & DND Mode)
REQ-ONBOARD-001	17. Additional	First-Time User Onboarding Flow & Role Profiling with Route Enforcement

7.0 DETAILED MODULE SPECIFICATIONS (MODULES 1 TO 6)

This section provides exhaustive, low-level technical specifications for all functional requirements across Modules 1 through 6. Every requirement defines the complete behavioral contract governing frontend components, backend controllers, domain services, database models, and security boundaries.

7.1 Module 1: Authentication and User Management

REQ-AUTH-001: User Registration via Two-Stage Staged Email OTP Verification

Purpose & Overview: Enables prospective users to create an account through a secure, two-stage verification process that blocks fake registrations and unverified accounts from polluting the permanent users table.

Authorized Actors & Roles: Public / Unauthenticated visitor.

Preconditions: Visitor has access to a valid, reachable email inbox; no existing permanent user exists with that email address.

Main Execution Workflow: 1. User navigates to /register. 2. Submits firstName, lastName, email, password, and optional phone. 3. System checks permanent users. 4. Hashes password with bcrypt. 5. Generates 6-digit cryptographic OTP and computes HMAC-SHA256 digest. 6. Upserts pending_registrations table with 5-minute expiry. 7. Dispatches email. 8. Client redirects to /verify-otp.

Alternative & Exceptional Workflows: If email already exists in users table, aborts immediately with HTTP 409 Conflict without dispatching email. If SMTP/Resend fails, aborts with HTTP 500.

Data I/O Contracts: Inputs: Inputs: { firstName, lastName, email, password, phone? } | Outputs: Outputs: HTTP 200 { message: 'Verification code sent', email }

Business Logic & Algorithmic Rules: Password must be hashed with bcrypt using cost factor 10. OTP generated via crypto.randomInt(100000, 1000000). OTP stored strictly as HMAC-SHA256 hash using server-side OTP_SECRET.

Validation Invariants: Email must conform to RFC 5322; password length >= 8 characters; phone (if provided) must match E.164 international format.

- **Backend Responsibilities:** authController.js calls otpService.js (initiateRegistration); interacts with mailService.js to deliver email via Resend API or Gmail SMTP.
- **Frontend Responsibilities:** Register.jsx renders registration form, handles input validation, animates Quackie emotion states (hasSuccess, isTyping), and navigates with route state.
- **Database Operations:** Reads users table to check duplication; writes pending_registrations (email, first_name, last_name, password_hash, otp_hash, otp_expires_at).
- **API Contracts & Routes:** POST /api/auth/register
- **Error Handling & Status Codes:** 400 Bad Request (Validation failure); 409 Conflict ('Email already registered'); 429 Too Many Requests (Rate limit exceeded); 500 Email Dispatch Error.
- **Module Dependencies:** Module 8 (Notifications/Mail Dispatcher), Module 1 (Users Model).

REQ-AUTH-002: Timing-Safe HMAC-SHA256 OTP Verification & Account Activation

Purpose & Overview: Verifies the 6-digit verification code using constant-time cryptographic comparison and atomically promotes the staged record to permanent user status.

Authorized Actors & Roles: Public / Unauthenticated prospective user holding a pending registration.

Preconditions: A valid, unexpired record exists in pending_registrations matching the submitted email.

Main Execution Workflow: 1. User enters 6-digit code on /verify-otp. 2. Backend computes HMAC-SHA256 of entered code. 3. Compares with stored otp_hash using crypto.timingSafeEqual. 4. Checks expiration. 5. Executes atomic Prisma transaction: deletes pending_registrations row, inserts new user into users with default role 'Team Member' and has_completed_onboarding: false. 6. Returns HTTP 201 Created. 7. Client redirects to /login.

Alternative & Exceptional Workflows: If code is incorrect, increments verification_attempts. If attempts reach 5, pending registration is invalidated. If expired, rejects with HTTP 400 ('Code expired').

Data I/O Contracts: Inputs: Inputs: { email, otp } | Outputs: Outputs: HTTP 201 { success: true, message: 'Email verified successfully', user }

Business Logic & Algorithmic Rules: Timing-safe comparison prevents side-channel timing attacks. Atomic transaction guarantees no orphaned pending registrations or duplicated users.

Validation Invariants: OTP must be exactly 6 digits; verification attempts < 5; current time < otp_expires_at.

- **Backend Responsibilities:** authController.js calls otpService.js (verifyOtpAndCreateUser); wraps DB operations in prisma.$transaction.
- **Frontend Responsibilities:** VerifyOtp.jsx provides 6 auto-advancing digit input boxes, handles clipboard paste, displays countdown timers, and shows error banners.
- **Database Operations:** Atomically deletes from pending_registrations and inserts into users (role_id -> roles.id where role_name = 'Team Member').
- **API Contracts & Routes:** POST /api/auth/verify-otp
- **Error Handling & Status Codes:** 400 Bad Request ('Invalid or expired code'); 404 Not Found ('No registration pending'); 429 Too Many Requests ('Maximum verification attempts exceeded').
- **Module Dependencies:** Module 1 (Registration), Database Transaction Engine.

REQ-AUTH-003: Registration OTP Resend with Enforced Cooldown

Purpose & Overview: Allows users to request a fresh OTP if the previous email was not received, enforcing strict cooldowns to prevent mail-bombing abuse.

Authorized Actors & Roles: Public / Unauthenticated prospective user holding a pending registration.

Preconditions: Record exists in pending_registrations.

Main Execution Workflow: 1. User clicks 'Resend Code'. 2. Backend checks last_resend_at. 3. Confirms >= 60 seconds have elapsed. 4. Generates new OTP and HMAC-SHA256 hash. 5. Resets verification_attempts to 0. 6. Extends expiry to now + 5 min. 7. Increments resend_count (max 5). 8. Dispatches fresh email. 9. Returns HTTP 200.

Alternative & Exceptional Workflows: If clicked before 60 seconds, rejects with HTTP 429 ('Please wait before requesting another code'). If resend_count >= 5, registration is terminated.

Data I/O Contracts: Inputs: Inputs: { email } | Outputs: Outputs: HTTP 200 { message: 'Verification code resent successfully' }

Business Logic & Algorithmic Rules: Sliding cooldown prevents automated mail spam while accommodating genuine delivery latency.

Validation Invariants: Cooldown >= 60 seconds; resend_count < 5.

- **Backend Responsibilities:** authController.js invokes otpService.js (resendRegistrationOtp).
- **Frontend Responsibilities:** VerifyOtp.jsx disables the resend button and renders an active 60-second visual countdown.
- **Database Operations:** Updates pending_registrations (otp_hash, otp_expires_at, last_resend_at, resend_count, verification_attempts).
- **API Contracts & Routes:** POST /api/auth/resend-otp
- **Error Handling & Status Codes:** 400 Bad Request ('Max resends exceeded'); 404 Not Found; 429 Too Many Requests ('Cooldown active').
- **Module Dependencies:** Module 1 (Registration), Mailer Service.

REQ-AUTH-004: User Login, 1-Day Signed JWT Issuance & Multi-Workspace Activity Logging

Purpose & Overview: Authenticates active users, generates signed JSON Web Tokens, and batch-records login audit events across all workspaces where the user holds membership.

Authorized Actors & Roles: Registered permanent user with is_active = true.

Preconditions: User account exists in users table and has verified email OTP.

Main Execution Workflow: 1. User enters email and password at /login. 2. Backend verifies bcrypt hash. 3. Confirms account is active. 4. Issues signed JWT with 1-day expiration. 5. Gathers all user workspaces from workspace_members. 6. Inserts 'LOGIN' audit records into activity_logs. 7. Returns JWT token and user profile object. 8. Client stores token in localStorage.

Alternative & Exceptional Workflows: If credentials invalid, increments failed attempt counter in rateLimiter. If unverified email, returns HTTP 401 instructing user to complete OTP verification.

Data I/O Contracts: Inputs: Inputs: { email, password } | Outputs: Outputs: HTTP 200 { token: string, user: { id, firstName, lastName, email, role, hasCompletedOnboarding } }

Business Logic & Algorithmic Rules: Passwords verified using bcrypt.compare. Batch activity logging executed via Prisma createMany for zero latency overhead.

Validation Invariants: Email must be non-empty valid format; password non-empty. Global IP limiter: 30 attempts/15 min; Account limiter: 5 attempts/15 min.

- **Backend Responsibilities:** authController.js calls authService.js (loginUser) and activityService.js (createBatchActivities).
- **Frontend Responsibilities:** Login.jsx manages input state, shows password toggle, validates required fields, updates AuthContext, and routes to / or /onboarding.
- **Database Operations:** Reads users, roles, workspace_members; writes batch records to activity_logs.
- **API Contracts & Routes:** POST /api/auth/login
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized ('Invalid email or password' / 'Please verify email'); 403 Forbidden ('Account deactivated'); 429 Rate Limit Exceeded.
- **Module Dependencies:** Module 8 (Activity Tracking), Module 5 (RBAC).

REQ-AUTH-005: Password Reset Request Link Dispatch

Purpose & Overview: Generates a time-bounded, cryptographically signed password reset token and dispatches recovery instructions via email.

Authorized Actors & Roles: Registered user who forgot credentials.

Preconditions: User account exists in users table.

Main Execution Workflow: 1. User submits email at /forgot-password. 2. Backend finds user. 3. Generates 15-minute signed JWT reset token incorporating user ID and password hash fingerprint. 4. Dispatches email with reset URL containing token. 5. Returns HTTP 200.

Alternative & Exceptional Workflows: To prevent user enumeration, if email does not exist, backend still returns HTTP 200 with generic success message.

Data I/O Contracts: Inputs: Inputs: { email } | Outputs: Outputs: HTTP 200 { message: 'Password reset instructions sent to your email' }

Business Logic & Algorithmic Rules: Token fingerprint incorporates existing password hash; changing password immediately invalidates any other outstanding reset tokens.

Validation Invariants: Valid email syntax; rate limited to 5 requests per hour.

- **Backend Responsibilities:** authController.js invokes authService.js (forgotPassword).
- **Frontend Responsibilities:** ForgotPassword.jsx collects email and displays confirmation banner.
- **Database Operations:** Reads users table.
- **API Contracts & Routes:** POST /api/auth/forgot-password
- **Error Handling & Status Codes:** 400 Bad Request; 429 Too Many Requests; 500 Mailer Error.
- **Module Dependencies:** Module 8 (Mail Dispatcher).

REQ-AUTH-006: Single-Use Password Reset Confirmation with Replay Mitigation

Purpose & Overview: Validates reset token, verifies token has not been previously consumed via used_password_reset_tokens table, and securely updates password.

Authorized Actors & Roles: User holding a valid password reset token.

Preconditions: Token signature is valid, unexpired, and token hash is absent from used_password_reset_tokens.

Main Execution Workflow: 1. User opens /reset-password?token=... 2. Submits new password. 3. Backend verifies JWT signature. 4. Computes SHA-256 hash of token string. 5. Checks used_password_reset_tokens table. 6. If absent, hashes new password with bcrypt. 7. Updates users.password_hash. 8. Inserts token hash into used_password_reset_tokens. 9. Invalidates active user JWTs via jwtCache.revokeUser. 10. Returns HTTP 200.

Alternative & Exceptional Workflows: If token hash exists in used_password_reset_tokens, rejects with HTTP 400 ('TOKEN_ALREADY_USED'). If expired, rejects with HTTP 400 ('TOKEN_EXPIRED').

Data I/O Contracts: Inputs: Inputs: { token, password } | Outputs: Outputs: HTTP 200 { message: 'Password reset successful' }

Business Logic & Algorithmic Rules: Atomic insertion into used_password_reset_tokens eliminates token replay attacks entirely.

Validation Invariants: Password length >= 8 characters; token signature valid.

- **Backend Responsibilities:** authController.js calls authService.js (resetPassword).
- **Frontend Responsibilities:** ResetPassword.jsx extracts token from query params, validates password length, and redirects to /login upon success.
- **Database Operations:** Updates users.password_hash; inserts into used_password_reset_tokens.
- **API Contracts & Routes:** POST /api/auth/reset-password
- **Error Handling & Status Codes:** 400 Bad Request ('Invalid or expired reset token'); 500 Internal Error.
- **Module Dependencies:** Module 1 (Users Model), Security Invalidation Layer.

REQ-AUTH-007: User Profile Retrieval and Contact Attribute Updates

Purpose & Overview: Allows authenticated users to inspect profile details and update name, phone, and avatar while maintaining email immutability.

Authorized Actors & Roles: Authenticated user.

Preconditions: User possesses valid Bearer JWT.

Main Execution Workflow: 1. Client requests GET /api/users/profile. 2. Backend returns user record (excluding password_hash). 3. User submits updates via PUT /api/users/profile. 4. Backend validates fields. 5. Updates PostgreSQL. 6. Returns updated profile.

Alternative & Exceptional Workflows: Attempts to alter email address are stripped or rejected to prevent identity hijacking.

Data I/O Contracts: Inputs: Inputs: { firstName, lastName, phone, profilePicture } | Outputs: Outputs: HTTP 200 { id, firstName, lastName, email, phone, profilePicture }

Business Logic & Algorithmic Rules: Email is immutable via profile update endpoint; changes require administrative authorization.

Validation Invariants: Names non-empty (<= 50 chars); phone matches international format; picture URL valid.

- **Backend Responsibilities:** userController.js calls userService.js.
- **Frontend Responsibilities:** Profile.jsx renders editable profile cards, avatar preview, and save buttons.
- **Database Operations:** Reads/Updates users table.
- **API Contracts & Routes:** GET /api/users/profile, PUT /api/users/profile
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 1 (Users Model).

REQ-AUTH-008: Authenticated Password Rotation

Purpose & Overview: Enables authenticated users to rotate their password after verifying their current password.

Authorized Actors & Roles: Authenticated user.

Preconditions: User possesses valid Bearer JWT.

Main Execution Workflow: 1. User enters currentPassword and newPassword at /profile/change-password. 2. Backend fetches user and compares currentPassword with bcrypt hash. 3. Hashes newPassword with bcrypt (10 rounds). 4. Updates users.password_hash. 5. Revokes active session tokens. 6. Returns HTTP 200.

Alternative & Exceptional Workflows: If currentPassword does not match, rejects with HTTP 400 ('Current password incorrect').

Data I/O Contracts: Inputs: Inputs: { currentPassword, newPassword } | Outputs: Outputs: HTTP 200 { message: 'Password updated successfully' }

Business Logic & Algorithmic Rules: New password must not be identical to current password.

Validation Invariants: New password length >= 8 characters.

- **Backend Responsibilities:** userController.js calls userService.js.
- **Frontend Responsibilities:** ChangePassword.jsx renders form with strength indicator and confirmation fields.
- **Database Operations:** Updates users.password_hash.
- **API Contracts & Routes:** POST /api/users/change-password
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized.
- **Module Dependencies:** Module 1 (Users Model).

REQ-AUTH-009: Interactive User Onboarding & First-Time Experience (FTE) Profiling

Purpose & Overview: Guides newly verified users through initial role selection and workspace orientation, persisting the completed state to PostgreSQL.

Authorized Actors & Roles: Newly verified user with has_completed_onboarding = false.

Preconditions: Authenticated user possessing valid JWT.

Main Execution Workflow: 1. ProtectedRoute intercepts user with hasCompletedOnboarding: false and redirects to /onboarding. 2. User completes multi-step questionnaire selecting role, experience, and primary use case. 3. Submits POST /api/users/onboarding. 4. Backend updates users.has_completed_onboarding = true. 5. Returns success. 6. Client updates AuthContext and routes to /.

Alternative & Exceptional Workflows: User cannot access main operational routes (/, /projects, /tasks) until onboarding completion flag is persisted.

Data I/O Contracts: Inputs: Inputs: { role, experience } | Outputs: Outputs: HTTP 200 { success: true, hasCompletedOnboarding: true }

Business Logic & Algorithmic Rules: Route guards enforce complete onboarding before granting access to project workflows.

Validation Invariants: Non-empty role selection.

- **Backend Responsibilities:** userController.js updates users.has_completed_onboarding.
- **Frontend Responsibilities:** FirstTimeExperience.jsx renders animated multi-step wizard; ProtectedRoute.jsx enforces redirect.
- **Database Operations:** Updates users.has_completed_onboarding.
- **API Contracts & Routes:** POST /api/users/onboarding
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized.
- **Module Dependencies:** Module 1 (Routing Guard).

REQ-AUTH-010: Adaptive Sliding-Window Rate Limiting

Purpose & Overview: Protects authentication endpoints from brute-force password guessing, credential stuffing, and registration spamming.

Authorized Actors & Roles: System middleware layer.

Preconditions: Incoming HTTP request to authentication endpoints.

Main Execution Workflow: 1. Request arrives at rateLimiter.js middleware. 2. Extracts client IP (using req.ip with trust proxy). 3. Evaluates token bucket for IP and account. 4. If request count exceeds ceiling within sliding window, request is aborted with HTTP 429. 5. If under threshold, request passes to next().

Alternative & Exceptional Workflows: Threshold breach sets Retry-After header and returns structured error JSON.

Data I/O Contracts: Inputs: Inputs: Client IP, email header/body. | Outputs: Outputs: Pass-through to next() or HTTP 429 Too Many Requests.

Business Logic & Algorithmic Rules: Sliding window in-memory token bucket. Automatically evicts expired timestamps to prevent memory leakage.

Validation Invariants: Limits: Global IP login (30/15min); Per-account login (5/15min); Register (10/hour); Forgot Password (5/hour).

- **Backend Responsibilities:** rateLimiter.js middleware mounted in app.js.
- **Frontend Responsibilities:** Handles HTTP 429 responses and displays user-friendly retry banners.
- **Database Operations:** None (in-memory Map storage).
- **API Contracts & Routes:** Mounted across all /api/auth/* endpoints.
- **Error Handling & Status Codes:** 429 Too Many Requests.
- **Module Dependencies:** Express Network Gateway.

REQ-AUTH-011: High-Performance In-Memory JWT Revocation & Validation LRU Cache

Purpose & Overview: Provides sub-millisecond JWT verification while supporting immediate token revocation when user permissions, roles, or active statuses change.

Authorized Actors & Roles: System security layer.

Preconditions: Authenticated request containing Authorization: Bearer <token>.

Main Execution Workflow: 1. authMiddleware.js extracts Bearer token. 2. Checks jwtCache. 3. If cached and valid, hydrates req.user instantly. 4. If not cached, verifies cryptographic signature with process.env.JWT_SECRET. 5. Checks if userId is in revoked list. 6. Queries user active status. 7. Caches verified payload. 8. Calls next().

Alternative & Exceptional Workflows: When an admin demotes a user or deactivates an account, jwtCache.revokeUser(userId) is triggered, causing immediate rejection on the user's next request.

Data I/O Contracts: Inputs: Inputs: HTTP Authorization Header. | Outputs: Outputs: Populated req.user object.

Business Logic & Algorithmic Rules: In-memory LRU cache reduces database hit rate for authentication checks by > 90%.

Validation Invariants: Token signature must be mathematically valid and unexpired.

- **Backend Responsibilities:** authMiddleware.js with internal jwtCache singleton.
- **Frontend Responsibilities:** Axios client injects token on every outgoing request.
- **Database Operations:** Reads users and roles only on cache miss.
- **API Contracts & Routes:** Applied to all protected /api/* routes.
- **Error Handling & Status Codes:** 401 Unauthorized ('Token missing or expired'); 403 Forbidden ('Token revoked').

- **Module Dependencies:** Global Middleware Pipeline.

7.2 Module 2: Workspace Management

REQ-WS-001: Workspace Creation & Automatic Owner Membership Provisioning

Purpose & Overview: Provisions a top-level tenant container and automatically establishes the creating user as the Workspace Owner.

Authorized Actors & Roles: Authenticated user.

Preconditions: User possesses valid JWT session.

Main Execution Workflow: 1. User clicks 'Create Workspace' at /workspaces. 2. Inputs name and description. 3. Backend creates row in workspaces. 4. Creates row in workspace_members setting workspace_role = 'Owner'. 5. Records 'CREATE' in activity_logs. 6. Returns workspace object.

Alternative & Exceptional Workflows: If name is missing or exceeds 100 characters, rejects with HTTP 400.

Data I/O Contracts: Inputs: Inputs: { name, description } | Outputs: Outputs: HTTP 201 { id, name, description, ownerId, workspaceRole: 'Owner' }

Business Logic & Algorithmic Rules: Every workspace must have exactly one designated Owner upon creation.

Validation Invariants: Name non-empty, length <= 100 characters.

- **Backend Responsibilities:** workspaceController.js calls workspaceService.js (createWorkspace).
- **Frontend Responsibilities:** Workspaces.jsx renders creation modal and appends new workspace to UI state.
- **Database Operations:** Inserts into workspaces and workspace_members; writes activity_logs.
- **API Contracts & Routes:** POST /api/workspaces
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized.
- **Module Dependencies:** Module 1 (Users), Module 8 (Activity Logs).

REQ-WS-002: Multi-Workspace Listing, Context Scoping & Active Switching

Purpose & Overview: Retrieves all workspaces where the user is an active member, allowing seamless operational context switching.

Authorized Actors & Roles: Authenticated user.

Preconditions: User possesses valid JWT session.

Main Execution Workflow: 1. User navigates to /workspaces or opens workspace dropdown in navbar. 2. Backend queries workspace_members where user_id = req.user.id. 3. Joins workspaces metadata and member counts. 4. Returns array of workspaces. 5. Client updates active workspace context.

Alternative & Exceptional Workflows: If user belongs to zero workspaces, client prompts user to create their first workspace.

Data I/O Contracts: Inputs: Inputs: None (derived from req.user.id) | Outputs: Outputs: HTTP 200 { workspaces: [{ id, name, description, memberCount, workspaceRole }] }

Business Logic & Algorithmic Rules: Users can only view workspaces they have explicitly been added to (unless holding Platform Admin role).

Validation Invariants: None.

- **Backend Responsibilities:** workspaceController.js calls workspaceService.js (getWorkspaces).
- **Frontend Responsibilities:** Workspaces.jsx, Navbar workspace switcher, and Sidebar context selectors.
- **Database Operations:** Joins workspaces and workspace_members.
- **API Contracts & Routes:** GET /api/workspaces
- **Error Handling & Status Codes:** 401 Unauthorized; 500 Internal Error.
- **Module Dependencies:** Module 1 (Users).

REQ-WS-004: Workspace Deletion & Cascading Tenant Record Purging

Purpose & Overview: Permanently destroys a workspace and cleanly cascades deletion across all child projects, tasks, dependencies, channels, and member associations.

Authorized Actors & Roles: Workspace Owner or Platform Admin.

Preconditions: Workspace exists and caller is verified Owner.

Main Execution Workflow: 1. Owner clicks 'Delete Workspace' and confirms warning prompt. 2. Backend verifies ownership. 3. Deletes workspace record from PostgreSQL. 4. Foreign key cascades purge child tables. 5. Returns HTTP 200.

Alternative & Exceptional Workflows: Workspace Admins and Members receive HTTP 403 Forbidden.

Data I/O Contracts: Inputs: Inputs: Workspace ID | Outputs: Outputs: HTTP 200 { message: 'Workspace deleted successfully' }

Business Logic & Algorithmic Rules: Strict cascading deletion invariant enforced at database engine level.

Validation Invariants: Target workspace ID must exist.

- **Backend Responsibilities:** workspaceController.js calls workspaceService.js (deleteWorkspace).
- **Frontend Responsibilities:** Workspaces.jsx provides confirmation dialog and removes deleted workspace from local state.
- **Database Operations:** Deletes from workspaces (cascades to workspace_members, projects, tasks, subtasks, task_dependencies, channels, messages).
- **API Contracts & Routes:** DELETE /api/workspaces/:id
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden ('Only workspace owner can delete workspace'); 404 Not Found.
- **Module Dependencies:** Database Engine Cascade Rules.

REQ-WS-006: Workspace Member Invitation with Duplicate Prevention

Purpose & Overview: Adds registered users to a workspace by user ID, enforcing composite uniqueness to prevent duplicate membership records.

Authorized Actors & Roles: Workspace Owner or Workspace Admin.

Preconditions: Target user exists in users table and is not currently a member of the workspace.

Main Execution Workflow: 1. Owner/Admin searches user via /api/search/users. 2. Selects user and role ('Admin' or 'Member'). 3. Submits POST /api/workspaces/:id/members. 4. Backend verifies unique constraint. 5. Inserts into workspace_members. 6. Emits socket notification. 7. Returns new member record.

Alternative & Exceptional Workflows: If user is already a member, PostgreSQL unique constraint uq_workspace_member intercepts write and returns HTTP 409 Conflict ('User already member').

Data I/O Contracts: Inputs: Inputs: { userId, workspaceRole } | Outputs: Outputs: HTTP 201 { id, workspaceId, userId, workspaceRole, joinedAt }

Business Logic & Algorithmic Rules: Composite unique index uq_workspace_member (workspace_id, user_id) guarantees referential integrity.

Validation Invariants: workspaceRole must be in ['Admin', 'Member']; target userId must exist.

- **Backend Responsibilities:** workspaceController.js calls workspaceService.js (addMember).
- **Frontend Responsibilities:** Workspaces.jsx members tab with user search autocomplete.
- **Database Operations:** Inserts into workspace_members.
- **API Contracts & Routes:** POST /api/workspaces/:id/members
- **Error Handling & Status Codes:** 400 Bad Request; 403 Forbidden; 409 Conflict ('User is already a member').
- **Module Dependencies:** Module 1 (Users), Module 8 (Notifications).

7.3 Module 3: Project Management

REQ-PROJ-001: Project Provisioning with Database Date Check Invariants

Purpose & Overview: Creates a project container within a workspace with validated start and end dates enforced both at the application and PostgreSQL storage engine levels.

Authorized Actors & Roles: Workspace Owner, Workspace Admin, or Platform Admin.

Preconditions: Parent workspace exists and caller holds managerial authority.

Main Execution Workflow: 1. User clicks 'Create Project' at /projects. 2. Enters title, description, category, priority, status, startDate, and endDate. 3. Backend verifies startDate <= endDate. 4. Inserts into projects. 5. Assigns creator as manager. 6. Emits WebSocket event project.created. 7. Returns project record.

Alternative & Exceptional Workflows: If startDate > endDate, backend halts execution and returns HTTP 400 Bad Request ('PROJECT_DATES_INVALID'). If bypassed, PostgreSQL check constraint chk_project_dates aborts insert.

Data I/O Contracts: Inputs: Inputs: { title, description, category, priority, status, startDate, endDate } | Outputs: Outputs: HTTP 201 Created Project Object

Business Logic & Algorithmic Rules: Database check constraint chk_project_dates enforces (start_date <= end_date).

Validation Invariants: Title non-empty (<= 150 chars); priority in ['Low', 'Medium', 'High', 'Critical']; status in ['Planning', 'In Progress', 'On Track', 'At Risk', 'Complete'].

- **Backend Responsibilities:** projectController.js calls projectService.js (createProject).
- **Frontend Responsibilities:** Projects.jsx modal form with calendar pickers and category selects.
- **Database Operations:** Inserts into projects (workspace_id, manager_id, title, ...); writes activity_logs.
- **API Contracts & Routes:** POST /api/workspaces/:workspaceId/projects
- **Error Handling & Status Codes:** 400 Bad Request ('Start date cannot be after end date'); 403 Forbidden; 404 Not Found.
- **Module Dependencies:** Module 2 (Workspaces), Database Check Constraints.

REQ-PROJ-002: Workspace Project Catalog with Dynamic Manager Resolution

Purpose & Overview: Displays active and archived projects within a workspace, dynamically resolving manager details from live workspace members.

Authorized Actors & Roles: Workspace members.

Preconditions: User belongs to the target workspace.

Main Execution Workflow: 1. User opens /projects. 2. Backend queries projects for workspaceId. 3. Frontend invokes getWorkspaceMembers(workspaceId). 4. Resolves manager names dynamically using getUser(project.managerId). 5. Computes completion progress from task states. 6. Renders project cards.

Alternative & Exceptional Workflows: If managerId is null or unassigned, renders 'Unassigned' badge cleanly without throwing runtime exceptions.

Data I/O Contracts: Inputs: Inputs: workspaceId query param, includeArchived bool | Outputs: Outputs: HTTP 200 { projects: [...] }

Business Logic & Algorithmic Rules: Dynamic resolution from workspace_members eliminates synthetic dummy manager names.

Validation Invariants: Valid workspaceId UUID.

- **Backend Responsibilities:** projectController.js calls projectService.js (getAll).
- **Frontend Responsibilities:** Projects.jsx renders responsive grid of project cards with progress bars and status badges.
- **Database Operations:** Reads projects, tasks count, and workspace_members.
- **API Contracts & Routes:** GET /api/workspaces/:workspaceId/projects
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden.
- **Module Dependencies:** Module 2 (Workspaces), Module 4 (Tasks).

REQ-PROJ-004: Project Archival and Restoration Lifecycle

Purpose & Overview: Enables soft-archival of projects to clean active operational views while preserving complete historical audit logs and analytics.

Authorized Actors & Roles: Workspace Owner, Workspace Admin, or Project Manager.

Preconditions: Project exists in active workspace.

Main Execution Workflow: 1. Manager clicks 'Archive Project'. 2. Backend updates projects.is_archived = true. 3. Project disappears from active lists. 4. If filter 'Show Archived' is toggled, project appears with 'Restore' action. 5. Restoring updates is_archived = false.

Alternative & Exceptional Workflows: Archiving a project does not delete tasks or files; all records remain queryable in compliance feeds.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { message: 'Project archived/restored successfully' }

Business Logic & Algorithmic Rules: Soft-delete flag preserves referential integrity across dependencies and activity logs.

Validation Invariants: Target project ID must exist.

- **Backend Responsibilities:** projectController.js calls projectService.js (archive / restore).
- **Frontend Responsibilities:** Projects.jsx provides archive/restore action buttons and view toggles.
- **Database Operations:** Updates projects.is_archived.
- **API Contracts & Routes:** PATCH /api/projects/:id/archive, PATCH /api/projects/:id/restore, DELETE /api/projects/:id
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden; 404 Not Found.
- **Module Dependencies:** Module 8 (Activity Tracking).

7.4 Module 4: Task Management and Assignment

REQ-TASK-001: Task Definition with Scheduling Constraints & Check Invariants

Purpose & Overview: Creates an actionable work item with designated priority, status, dates, estimates, and assignees, validated by database check constraints.

Authorized Actors & Roles: Workspace or Project member.

Preconditions: Parent project exists and caller has access.

Main Execution Workflow: 1. User opens Create Task dialog. 2. Enters title, description, priority, status, startDate, dueDate, estimatedHours, assignedTo. 3. Backend verifies startDate <= dueDate. 4. Inserts into tasks. 5. Emits socket event task.created. 6. Dispatches notification to assignee. 7. Returns task object.

Alternative & Exceptional Workflows: If startDate > dueDate, rejected with HTTP 400 Bad Request ('TASK_DATES_INVALID'). PostgreSQL constraint chk_task_dates provides secondary safety barrier.

Data I/O Contracts: Inputs: Inputs: { title, description, priority, status, startDate, dueDate, estimatedHours, assignedTo } | Outputs: Outputs: HTTP 201 Created Task Object

Business Logic & Algorithmic Rules: PostgreSQL check constraint chk_task_dates enforces (start_date <= due_date).

Validation Invariants: Title required (<= 200 chars); priority in ['Low', 'Medium', 'High', 'Critical']; status in ['Backlog', 'To Do', 'In Progress', 'Review', 'Completed'].

- **Backend Responsibilities:** taskController.js calls taskService.js (createTask).

- **Frontend Responsibilities:** Tasks.jsx, Kanban.jsx Create Task modals with real-time field validation.
- **Database Operations:** Inserts into tasks; inserts into notifications if assignedTo is populated.
- **API Contracts & Routes:** POST /api/projects/:projectId/tasks
- **Error Handling & Status Codes:** 400 Bad Request; 403 Forbidden; 404 Not Found.
- **Module Dependencies:** Module 3 (Projects), Module 8 (Notifications).

REQ-TASK-002: Paginated Task Listing with Multi-Criteria Filtering & Bounding

Purpose & Overview: Explores project tasks with flexible search, priority, and status filters, strictly bounding maximum page size to prevent memory exhaustion.

Authorized Actors & Roles: Project members.

Preconditions: Project exists.

Main Execution Workflow: 1. Client requests GET /api/projects/:projectId/tasks?page=1&limit=20. 2. Backend parses filters. 3. Enforces limit <= 100. 4. Queries tasks using composite indexes. 5. Returns items, totalCount, totalPages, and page.

Alternative & Exceptional Workflows: If client requests limit > 100, backend rejects request with HTTP 400 Bad Request ('Limit cannot exceed 100').

Data I/O Contracts: Inputs: Inputs: Query params { page, limit, status, priority, assignedTo, search } | Outputs: Outputs: HTTP 200 { tasks: [...], pagination: { totalCount, totalPages, page } }

Business Logic & Algorithmic Rules: Query bounded to prevent DoS memory spikes. Composite index idx_tasks_project_active_created optimizes high-concurrency sorting.

Validation Invariants: limit integer between 1 and 100.

- **Backend Responsibilities:** taskController.js calls taskService.js (getAll).
- **Frontend Responsibilities:** Tasks.jsx table view with pagination bar and filter dropdowns.
- **Database Operations:** Reads tasks table with indexed filters.
- **API Contracts & Routes:** GET /api/projects/:projectId/tasks
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Database Indexing Engine.

REQ-TASK-006: Task Duplication (Deep Cloning with Subtasks)

Purpose & Overview: Duplicates an existing task into a fresh active task, copying descriptions, estimates, and checklist subtasks without corrupting graph dependencies.

Authorized Actors & Roles: Project contributors.

Preconditions: Target task exists.

Main Execution Workflow: 1. User clicks 'Duplicate Task'. 2. Backend reads original task and associated subtasks. 3. Clones task with title suffix '(Copy)', status reset to 'To Do', and fresh UUID. 4. Clones all subtasks under new task ID. 5. Returns cloned task object.

Alternative & Exceptional Workflows: Dependencies are deliberately not cloned to prevent accidental circular graph cycles.

Data I/O Contracts: Inputs: Inputs: Task ID | Outputs: Outputs: HTTP 201 Cloned Task Object with cloned subtasks

Business Logic & Algorithmic Rules: Deep clone executed within atomic Prisma transaction.

Validation Invariants: Valid task UUID.

- **Backend Responsibilities:** taskController.js calls taskService.js (duplicate).
- **Frontend Responsibilities:** Tasks.jsx action menu with duplicate option.
- **Database Operations:** Reads tasks and subtasks; inserts new rows into tasks and subtasks.
- **API Contracts & Routes:** POST /api/tasks/:id/duplicate
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 5 (Subtasks).

REQ-TASK-008: Smart Task Prioritization Algorithm

Purpose & Overview: Ranks an engineer's assigned tasks algorithmically based on urgency, severity, slack, and downstream dependency blockers.

Authorized Actors & Roles: Authenticated user.

Preconditions: User has assigned tasks.

Main Execution Workflow: 1. User requests GET /api/tasks/prioritized. 2. Backend gathers active tasks for user. 3. Evaluates: Priority_Score = (Urgency_Factor * 0.4) + (Severity_Weight * 0.3) + (Dependency_Block_Factor * 0.3). 4. Returns ordered tasks.

Alternative & Exceptional Workflows: Completed or archived tasks are automatically excluded from prioritization calculations.

Data I/O Contracts: Inputs: Inputs: req.user.id | Outputs: Outputs: HTTP 200 { prioritizedTasks: [...] }

Business Logic & Algorithmic Rules: Mathematical weighted ranking prioritizing tasks that unblock multiple downstream teammates.

Validation Invariants: None.

- **Backend Responsibilities:** taskController.js calls taskPriorityService.js.
- **Frontend Responsibilities:** Tasks.jsx 'Smart Sort' toggle and Quackie Focus Mission widget.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** GET /api/tasks/prioritized
- **Error Handling & Status Codes:** 401 Unauthorized.
- **Module Dependencies:** Module 5 (Dependencies).

7.5 Module 5: Subtasks and Dependencies

REQ-DEP-001: Subtask Decomposition & Progress Calculation

Purpose & Overview: Breaks down parent tasks into checklist items and dynamically calculates task completion percentage based on finished items.

Authorized Actors & Roles: Project contributors.

Preconditions: Parent task exists.

Main Execution Workflow: 1. User opens task drawer and types subtask title. 2. Backend inserts row into subtasks. 3. When toggled to 'Done', parent task updates subtask completion progress: Progress % = (Done Subtasks / Total Subtasks) * 100.

Alternative & Exceptional Workflows: Deleting a parent task cascades deletion to all associated subtasks.

Data I/O Contracts: Inputs: Inputs: { title, dueDate } | Outputs: Outputs: HTTP 201 Created Subtask Object

Business Logic & Algorithmic Rules: Progress is computed dynamically, avoiding stale database integer counters.

Validation Invariants: Title non-empty (<= 200 chars); status in ['To Do', 'In Progress', 'Done'].

- **Backend Responsibilities:** subtaskController.js calls subtaskService.js.
- **Frontend Responsibilities:** SubtaskSection.jsx inside task drawer with checkable boxes and progress meter.
- **Database Operations:** Inserts/Updates/Deletes subtasks (task_id -> tasks.id ON DELETE CASCADE).
- **API Contracts & Routes:** POST /api/tasks/:taskId/subtasks, PATCH /api/subtasks/:id/status, DELETE /api/subtasks/:id
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks).

REQ-DEP-003: Directed Dependency Edge Creation with Kahn's Algorithm Cycle Prevention

Purpose & Overview: Establishes a directed dependency edge (taskId depends on dependsOnTaskId) while preventing circular deadlocks using topological sorting.

Authorized Actors & Roles: Project contributors.

Preconditions: Both tasks belong to the same project; neither task is currently linked in reverse.

Main Execution Workflow: 1. User selects prerequisite task in dependency drawer. 2. Submits POST /api/tasks/:taskId/dependencies. 3. Backend loads project dependency DAG. 4. Adds candidate edge in memory. 5. Runs Kahn's Algorithm. 6. If topological sort visits all nodes, edge is acyclic -> writes to PostgreSQL. 7. Returns HTTP 201.

Alternative & Exceptional Workflows: If adding edge creates a directed cycle (e.g. A -> B -> C -> A), Kahn's algorithm detects unprocessed nodes with non-zero in-degrees. Transaction is aborted and returns HTTP 409 Conflict ('DEPENDENCY_CYCLE').

Data I/O Contracts: Inputs: Inputs: { dependsOnTaskId } | Outputs: Outputs: HTTP 201 { task_id, depends_on_task_id, createdAt }

Business Logic & Algorithmic Rules: Kahn's Algorithm ensures the project dependency graph remains a strict Directed Acyclic Graph (DAG) at all times.

Validation Invariants: Self-dependency (A -> A) rejected with HTTP 400. Cross-project dependency rejected with HTTP 400. Cycles rejected with HTTP 409.

- **Backend Responsibilities:** taskDependencyController.js calls taskDependencyService.js (createDependency).
- **Frontend Responsibilities:** TaskDependencySection.jsx with dependency selector and cycle error toast alerts.
- **Database Operations:** Inserts into task_dependencies [task_id, depends_on_task_id].
- **API Contracts & Routes:** POST /api/tasks/:taskId/dependencies
- **Error Handling & Status Codes:** 400 Bad Request ('Self dependency prohibited'); 409 Conflict ('Circular dependency detected'); 404 Not Found.
- **Module Dependencies:** Graph Theory Engine, Module 4 (Tasks).

REQ-DEP-005: Task Dependency Introspection & Blocked Status Detection

Purpose & Overview: Evaluates whether a task is executable or currently blocked by incomplete upstream prerequisites.

Authorized Actors & Roles: Project members.

Preconditions: Task exists.

Main Execution Workflow: 1. Client requests GET /api/tasks/:taskId/dependencies. 2. Backend queries task_dependencies. 3. Gathers upstream prerequisite tasks (blockedBy) and downstream tasks (blocking). 4. Evaluates: isBlocked = any(upstream.status !== 'Completed'). 5. Returns dependency analysis.

Alternative & Exceptional Workflows: When all upstream tasks reach 'Completed' status, isBlocked resolves to false.

Data I/O Contracts: Inputs: Inputs: Task ID | Outputs: Outputs: HTTP 200 { taskId, isBlocked, blockedBy: [...], blocking: [...] }

Business Logic & Algorithmic Rules: Real-time evaluation ensures instant status unblocking without manual supervisor intervention.

Validation Invariants: Valid task UUID.

- **Backend Responsibilities:** taskDependencyController.js calls taskDependencyService.js (getTaskDependencies).
- **Frontend Responsibilities:** TaskDependencySection.jsx renders visual 'Blocked' warning badges on task cards.
- **Database Operations:** Reads task_dependencies and tasks.
- **API Contracts & Routes:** GET /api/tasks/:taskId/dependencies
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks).

7.6 Module 6: Kanban Board and Calendar System

REQ-KAN-001: 5-Column Interactive Agile Kanban Board

Purpose & Overview: Organizes tasks visually across standard agile workflow stages with real-time status partitioning.

Authorized Actors & Roles: Project contributors.

Preconditions: Project exists and user has access.

Main Execution Workflow: 1. User navigates to /kanban. 2. Backend queries active tasks for project. 3. Frontend partitions tasks into 5 columns: Backlog, To Do, In Progress, Review, Completed. 4. Displays cards with priority tags, avatars, dates, and blocked chips.

Alternative & Exceptional Workflows: Archived tasks are excluded from board columns.

Data I/O Contracts: Inputs: Inputs: projectId query param | Outputs: Outputs: HTTP 200 array of tasks

Business Logic & Algorithmic Rules: Status columns correspond strictly to database task status values.

Validation Invariants: Valid projectId.

- **Backend Responsibilities:** taskController.js (getAll).
- **Frontend Responsibilities:** Kanban.jsx, KanbanColumn.jsx, KanbanTaskCard.jsx.
- **Database Operations:** Reads tasks table.
- **API Contracts & Routes:** GET /api/projects/:projectId/tasks
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks).

REQ-KAN-002: Optimistic Drag-and-Drop Status Transitions with Automatic Rollback

Purpose & Overview: Provides low-latency status transitions when dragging cards between columns, guaranteeing automatic state rollback if the server rejects the move.

Authorized Actors & Roles: Project contributors.

Preconditions: User is actively viewing Kanban board.

Main Execution Workflow: 1. User drags card from 'In Progress' to 'Completed'. 2. Frontend immediately updates local state (optimistic update). 3. Dispatches PATCH /api/tasks/:id/status. 4. Backend validates transition, writes to DB, emits WebSocket broadcast. 5. If successful, card remains in new column.

Alternative & Exceptional Workflows: If backend returns error (e.g. network disconnect or blocked task guard), client catch block executes setTasks(previousTasks) and renders error toast.

Data I/O Contracts: Inputs: Inputs: { status: 'Completed' } | Outputs: Outputs: HTTP 200 Updated Task Object

Business Logic & Algorithmic Rules: Optimistic UI guarantees zero visual lag; state rollback guarantees eventual consistency.

Validation Invariants: Status must be valid enum value.

- **Backend Responsibilities:** taskController.js calls taskService.js (updateStatus).
- **Frontend Responsibilities:** Kanban.jsx drag event handlers (onDragStart, onDragOver, onDrop) with rollback state preservation.
- **Database Operations:** Updates tasks.status.
- **API Contracts & Routes:** PATCH /api/tasks/:id/status

- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found; 500 Network Error.
- **Module Dependencies:** Module 4 (Tasks), WebSocket Gateway.

REQ-KAN-004: Date-Bounded Calendar Schedule View & Chronological Filtering

Purpose & Overview: Displays time-allocated tasks and project milestones across Month, Week, and Agenda views with strict date range validation.

Authorized Actors & Roles: Workspace members.

Preconditions: Workspace contains tasks or projects with valid dates.

Main Execution Workflow: 1. User opens /calendar?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD. 2. Backend parses bounds via parseCalendarBounds. 3. Validates startDate <= endDate. 4. Fetches tasks (filtering start_date or due_date) and projects. 5. Returns combined event array. 6. Calendar renders events on month grid.

Alternative & Exceptional Workflows: If startDate > endDate, backend rejects request with HTTP 400 Bad Request ('CALENDAR_RANGE_INVALID').

Data I/O Contracts: Inputs: Inputs: Query params { startDate, endDate } | Outputs: Outputs: HTTP 200 { events: [{ id, title, type: 'TASK'|'PROJECT', start, end, priority, status }] }

Business Logic & Algorithmic Rules: Bounds checking prevents infinite date iteration in calendar rendering engines.

Validation Invariants: Valid ISO date strings (YYYY-MM-DD); startDate <= endDate.

- **Backend Responsibilities:** calendarController.js calls calendarService.js (getCalendarEvents).
- **Frontend Responsibilities:** Calendar.jsx month grid with event pills and date selection drawers.
- **Database Operations:** Reads tasks and projects.
- **API Contracts & Routes:** GET /api/calendar
- **Error Handling & Status Codes:** 400 Bad Request ('Start date cannot be after end date'); 401 Unauthorized.
- **Module Dependencies:** Module 3 (Projects), Module 4 (Tasks).

7.0 DETAILED MODULE SPECIFICATIONS (MODULES 7 TO 12)

This section documents the formal engineering specifications for Modules 7 through 12, detailing collaboration comments, authorized streaming attachments, automated background sweeps, multi-metric dashboards, global search, risk matrices, and architectural decision records.

7.7 Module 7: Comments and File Attachments

REQ-FILE-001: Task Comment Thread Creation and Rich Discussion Logging

Purpose & Overview: Enables team members to collaborate on tasks by posting threaded discussion notes, preserving chronological conversation records.

Authorized Actors & Roles: Project contributors and workspace members.

Preconditions: Parent task exists and user has project access.

Main Execution Workflow: 1. User enters text in comment box inside task drawer. 2. Clicks 'Post Comment'. 3. Backend verifies non-empty content. 4. Inserts into comments table. 5. Dispatches notification to task assignee. 6. Returns comment object. 7. Frontend appends comment with relative timestamp.

Alternative & Exceptional Workflows: If comment content is blank, frontend and backend reject request with HTTP 400 Bad Request.

Data I/O Contracts: Inputs: Inputs: { content: string } | Outputs: Outputs: HTTP 201 { id, taskId, userId, content, createdAt, user: { firstName, lastName, profilePicture } }

Business Logic & Algorithmic Rules: Comments are tied to parent tasks via foreign key cascade; deleting a task purges associated comments.

Validation Invariants: Content must be non-empty string.

- **Backend Responsibilities:** commentController.js calls commentService.js (createComment).
- **Frontend Responsibilities:** CommentSection.jsx inside task drawer with auto-expanding textarea and user avatar cards.
- **Database Operations:** Inserts into comments; inserts into notifications if commenter != assignee.
- **API Contracts & Routes:** POST /api/tasks/:taskId/comments, GET /api/tasks/:taskId/comments
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks), Module 8 (Notifications).

REQ-FILE-002: Comment Author Authorization & Moderation Guard

Purpose & Overview: Restricts comment editing and deletion strictly to the original author, while granting administrative moderation override to Platform Admins.

Authorized Actors & Roles: Comment author or Platform Admin.

Preconditions: Target comment exists.

Main Execution Workflow: 1. User clicks 'Edit' or 'Delete' on a comment. 2. Backend compares req.user.id with comment.user_id. 3. If caller is author or Admin, update/delete executes. 4. Returns HTTP 200.

Alternative & Exceptional Workflows: If a non-author user attempts to edit or delete another user's comment, request is rejected with HTTP 403 Forbidden ('Not authorized to modify this comment').

Data I/O Contracts: Inputs: Inputs: Comment ID, updated content | Outputs: Outputs: HTTP 200 Updated Comment / Deletion Confirmation

Business Logic & Algorithmic Rules: Strict author ownership invariant prevents unauthorized modification of discussion audit history.

Validation Invariants: Valid comment UUID.

- **Backend Responsibilities:** commentController.js calls commentService.js (updateComment / deleteComment).
- **Frontend Responsibilities:** CommentSection.jsx conditionally renders edit/delete buttons only for author's own comments.
- **Database Operations:** Updates / Deletes from comments table.
- **API Contracts & Routes:** PUT /api/comments/:id, PATCH /api/comments/:id, DELETE /api/comments/:id
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden ('Unauthorized'); 404 Not Found.
- **Module Dependencies:** Module 1 (Users), Module 5 (RBAC).

REQ-FILE-003: Multipart File Upload via Multer with UUID File Renaming

Purpose & Overview: Handles physical file uploads using Multer multipart processing, storing files on disk with cryptographic UUID filenames and validating file extensions.

Authorized Actors & Roles: Project contributors.

Preconditions: Target task or project exists and caller possesses upload permissions.

Main Execution Workflow: 1. User selects file in upload dialog. 2. Client sends multipart/form-data. 3. Multer intercepts request, checks size (<= 25MB). 4. Validates MIME type against blocked executable extensions (.exe, .sh, .bat, .js). 5. Generates UUID filename (e.g. 8d3a-4c2e.pdf) on disk. 6. Inserts metadata into attachments table. 7. Returns attachment record.

Alternative & Exceptional Workflows: If file exceeds 25MB or extension is in blocklist, Multer rejects request with HTTP 400 Bad Request ('FILE_TYPE_BLOCKED' or 'FILE_TOO_LARGE').

Data I/O Contracts: Inputs: Inputs: Multipart file, taskId or projectId | Outputs: Outputs: HTTP 201 { id, fileName, fileUrl, fileType, fileSize, uploadedBy }

Business Logic & Algorithmic Rules: Files renamed to UUIDs on disk to eliminate path traversal and filename collision vulnerabilities.

Validation Invariants: File size <= 25MB; extension not in ['.exe', '.sh', '.bat', '.cmd', '.js', '.vbs'].

- **Backend Responsibilities:** uploadMiddleware.js and attachmentController.js.
- **Frontend Responsibilities:** FileSection.jsx, TaskAttachmentSection.jsx with drag-and-drop file dropzone.
- **Database Operations:** Inserts into attachments; writes physical file to uploads/attachments/ directory.
- **API Contracts & Routes:** POST /api/tasks/:taskId/attachments, POST /api/projects/:projectId/attachments
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 413 Payload Too Large.
- **Module Dependencies:** Storage Engine, Middleware Pipeline.

REQ-FILE-005: Tenant-Isolated Authorized Streaming File Downloads

Purpose & Overview: Streams file downloads securely after verifying the requesting user belongs to the specific project and workspace owning the file.

Authorized Actors & Roles: Authenticated members of the target project/workspace.

Preconditions: Attachment record exists and physical file is present on disk.

Main Execution Workflow: 1. User clicks 'Download' link (/api/attachments/:id/download). 2. Backend queries attachment metadata. 3. Verifies req.user.id is an active member of parent project/workspace. 4. Sets Content-Disposition header with original file name. 5. Streams file buffer to client via res.download().

Alternative & Exceptional Workflows: If requesting user does not hold membership in the workspace owning the file, request is rejected with HTTP 403 Forbidden ('Access denied to this file').

Data I/O Contracts: Inputs: Inputs: Attachment ID | Outputs: Outputs: Binary file stream with original filename

Business Logic & Algorithmic Rules: Eliminates Insecure Direct Object References (IDOR) by coupling download authorization directly to database tenant membership.

Validation Invariants: Valid attachment UUID.

- **Backend Responsibilities:** attachmentController.js (downloadAttachment).
- **Frontend Responsibilities:** FileSection.jsx download action button triggering authenticated download.
- **Database Operations:** Reads attachments, projects, and workspace_members.
- **API Contracts & Routes:** GET /api/attachments/:id/download
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden ('Tenant boundary violation'); 404 Not Found.
- **Module Dependencies:** Module 2 (Workspaces), Module 3 (Projects).

REQ-FILE-006: Direct /uploads Static Access Prohibition

Purpose & Overview: Blocks unauthenticated or direct browser access to physical upload directories, requiring all asset reads to pass through authorized endpoints.

Authorized Actors & Roles: System security layer.

Preconditions: Incoming HTTP request to /uploads/* path.

Main Execution Workflow: 1. Request hits app.use('/uploads', ...). 2. Route handler immediately intercepts request. 3. Aborts and returns HTTP 403 Forbidden with structured JSON error.

Alternative & Exceptional Workflows: Direct image hotlinking and unauthenticated document scraping are completely blocked.

Data I/O Contracts: Inputs: Inputs: HTTP GET /uploads/... | Outputs: Outputs: HTTP 403 { success: false, message: 'Direct file access is disabled. Use /api/attachments/:id/download' }

Business Logic & Algorithmic Rules: Enforces zero-trust storage isolation: physical disk directory is never exposed statically.

Validation Invariants: Path prefix matches /uploads.

- **Backend Responsibilities:** Express app.js static blocker route.
- **Frontend Responsibilities:** Frontend renders files using authorized download endpoints or object URLs.
- **Database Operations:** None.

- **API Contracts & Routes:** ALL requests matching /uploads/*
- **Error Handling & Status Codes:** 403 Forbidden.
- **Module Dependencies:** Express Network Gateway.

7.8 Module 8: Notifications and Activity Tracking

REQ-NOTIF-001: Real-Time In-App Notification Feed & Unread Counter Badge

Purpose & Overview: Presents user-directed alerts for task assignments, mentions, and deadline warnings with unread counters and popover feeds.

Authorized Actors & Roles: Authenticated user.

Preconditions: User is logged in.

Main Execution Workflow: 1. Navbar bell component fetches unread count via GET /api/notifications/unread-count. 2. Displays red badge if unread > 0. 3. User clicks bell to open popover. 4. Fetches recent alerts via GET /api/notifications. 5. Displays alerts with severity icons and timestamps.

Alternative & Exceptional Workflows: Incoming WebSocket alerts increment the badge counter instantly without requiring manual page reload.

Data I/O Contracts: Inputs: Inputs: req.user.id | Outputs: Outputs: HTTP 200 { notifications: [...], unreadCount: number }

Business Logic & Algorithmic Rules: Alerts partitioned per user with composite unique deduplication keys.

Validation Invariants: None.

- **Backend Responsibilities:** notificationController.js calls notificationService.js.
- **Frontend Responsibilities:** NotificationBell.jsx popover component in header with unread indicator badge.
- **Database Operations:** Reads notifications (user_id, is_read, dedupe_key, created_at).
- **API Contracts & Routes:** GET /api/notifications, GET /api/notifications/unread-count
- **Error Handling & Status Codes:** 401 Unauthorized.
- **Module Dependencies:** Module 1 (Users), WebSocket Gateway.

REQ-NOTIF-003: Automated Daily Approaching Deadline Background Sweeper

Purpose & Overview: Automatically identifies tasks due within 24 to 48 hours and creates deduplicated deadline warning notifications.

Authorized Actors & Roles: Backend automated background daemon (Node-Cron).

Preconditions: Cron job running on cluster primary process at 09:00 UTC daily.

Main Execution Workflow: 1. Node-cron triggers dueDateNotificationService.js at 09:00 UTC ('0 9 * * *'). 2. Queries active tasks where due_date is within [now, now + 48 hours]. 3. Filters uncompleted tasks. 4. Generates dedupe_key: deadline:<taskId>:<date>. 5. Inserts notification into notifications table. 6. Emits socket alert. 7. Dispatches email reminder.

Alternative & Exceptional Workflows: If a notification with the same dedupe_key already exists, PostgreSQL composite unique constraint idx_notifications_dedupe_unique skips insert, preventing alert spamming.

Data I/O Contracts: Inputs: Inputs: System clock / scheduled trigger | Outputs: Outputs: Created notifications and dispatched emails

Business Logic & Algorithmic Rules: Runs exclusively on cluster primary process to prevent duplicate execution across worker nodes.

Validation Invariants: Task status !== 'Completed'; due_date within 48h.

- **Backend Responsibilities:** server.js cron scheduler and dueDateNotificationService.js.
- **Frontend Responsibilities:** NotificationBell.jsx and Dashboard upcoming deadline widgets.
- **Database Operations:** Reads tasks; writes notifications (enforcing dedupe_key unique index).
- **API Contracts & Routes:** Background Scheduled Process
- **Error Handling & Status Codes:** Logs errors to server console without crashing worker processes.
- **Module Dependencies:** Module 4 (Tasks), Node-Cron, Mailer Service.

REQ-NOTIF-006: Immutable Workspace-Wide Activity Audit Logging

Purpose & Overview: Maintains an immutable, append-only operational audit trail capturing user actions across projects, tasks, and workspaces.

Authorized Actors & Roles: All authenticated users during operational actions.

Preconditions: Operational action (create, update, status change, login) executed successfully.

Main Execution Workflow: 1. Controller/service invokes activityService.js. 2. Constructs payload: { workspaceId, userId, actionType, entityType, entityId, description }. 3. Inserts into activity_logs table. 4. Emits activity.logged socket event.

Alternative & Exceptional Workflows: Audit logs cannot be updated or deleted by any user or administrator through API routes (strictly append-only).

Data I/O Contracts: Inputs: Inputs: Activity event payload | Outputs: Outputs: Appended activity_logs record

Business Logic & Algorithmic Rules: Audit log entries provide historical evidence used by standup generators and compliance feeds.

Validation Invariants: actionType in ['CREATE', 'UPDATE', 'STATUS_CHANGE', 'DELETE', 'LOGIN']; entityType in ['TASK', 'PROJECT', 'WORKSPACE', 'USER'].

- **Backend Responsibilities:** activityService.js (createActivity, createBatchActivities).
- **Frontend Responsibilities:** ActivityTimeline.jsx, AdminActivityFeed.jsx.
- **Database Operations:** Inserts into activity_logs table.
- **API Contracts & Routes:** GET /api/workspaces/:workspaceId/activities, GET /api/admin/activity
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden.
- **Module Dependencies:** Module 2 (Workspaces), Module 4 (Tasks).

7.9 Module 9: Dashboards, Reports, and Productivity Analytics

REQ-DASH-001: Personal Operations Command Dashboard

Purpose & Overview: Presents an executive summary of personal tasks, approaching deadlines, and active project health in a responsive layout.

Authorized Actors & Roles: Authenticated user.

Preconditions: User has active session.

Main Execution Workflow: 1. User navigates to / (root). 2. Dashboard fetches assigned tasks via /api/tasks/my-tasks. 3. Fetches workspace projects. 4. Computes Stat Cards (Total Assigned, Completed, In-Progress, Overdue). 5. Displays Upcoming Deadlines list and Project Progress meters.

Alternative & Exceptional Workflows: If user belongs to zero workspaces, shows zero-state prompting workspace creation.

Data I/O Contracts: Inputs: Inputs: req.user.id | Outputs: Outputs: HTTP 200 aggregated task and project lists

Business Logic & Algorithmic Rules: Client aggregates metrics in real time from live task arrays without synthetic filler numbers.

Validation Invariants: None.

- **Backend Responsibilities:** taskController.js (getMine), projectController.js (getAll).
- **Frontend Responsibilities:** Dashboard.jsx, StatCard.jsx, DashboardTaskRow.jsx, UpcomingItem.jsx.
- **Database Operations:** Reads tasks and projects.
- **API Contracts & Routes:** GET /api/tasks/my-tasks, GET /api/workspaces/:workspaceId/projects
- **Error Handling & Status Codes:** 401 Unauthorized.
- **Module Dependencies:** Module 3 (Projects), Module 4 (Tasks).

REQ-DASH-002: Project Analytics & Reports Dashboard

Purpose & Overview: Computes operational velocity, task completion rate, overdue volume, and multi-dimensional status distributions for projects.

Authorized Actors & Roles: Workspace members.

Preconditions: Target workspace and optional project exist.

Main Execution Workflow: 1. User opens /reports. 2. Backend executes reportService.js. 3. Aggregates tasks: Completion Rate = (Completed / Total) * 100. 4. Computes pending vs overdue counts. 5. Groups counts by priority and status. 6. Returns structured analytics JSON. 7. Frontend renders charts and metrics.

Alternative & Exceptional Workflows: Archived tasks are excluded from calculation to preserve active operational accuracy.

Data I/O Contracts: Inputs: Inputs: Query params { projectId?, workspaceId? } | Outputs: Outputs: HTTP 200 { completion: {...}, pendingOverdue: {...}, byPriority: [...], byStatus: [...] }

Business Logic & Algorithmic Rules: Mathematical aggregation running directly over PostgreSQL relational tables.

Validation Invariants: Valid workspace or project UUID.

- **Backend Responsibilities:** reportController.js calls reportService.js (getDashboardMetrics).
- **Frontend Responsibilities:** Reports.jsx with completion meters, overdue indicators, and distribution bars.
- **Database Operations:** Aggregates over tasks and projects.
- **API Contracts & Routes:** GET /api/reports/dashboard
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks).

REQ-DASH-003: Productivity Analytics, Focus Index & Velocity Trends

Purpose & Overview: Calculates personal engineering throughput, cycle times from task creation to completion, focus consistency, and rolling completion trends.

Authorized Actors & Roles: Authenticated user.

Preconditions: User has created or completed tasks.

Main Execution Workflow: 1. User opens /productivity. 2. Backend analyzes tasks where assigned_to = req.user.id. 3. Calculates cycle time: Duration = completion_timestamp - created_at. 4. Computes Focus Index (0-100 score). 5. Calculates 14-day rolling completion velocity. 6. Returns metrics payload.

Alternative & Exceptional Workflows: If user has zero completed tasks, displays clean zero-state guidance.

Data I/O Contracts: Inputs: Inputs: req.user.id | Outputs: Outputs: HTTP 200 { cycleTimeHours, cycleTimeDays, focusIndex, velocityTrend: [...] }

Business Logic & Algorithmic Rules: Evaluates uninterrupted execution blocks to derive the Focus Index.

Validation Invariants: None.

- **Backend Responsibilities:** productivityController.js calls productivityService.js.
- **Frontend Responsibilities:** Productivity.jsx page with metric tiles and 14-day activity trend graph.
- **Database Operations:** Analyzes tasks created_at, updated_at, status.
- **API Contracts & Routes:** GET /api/productivity
- **Error Handling & Status Codes:** 401 Unauthorized.
- **Module Dependencies:** Module 4 (Tasks).

REQ-DASH-004: 7-Dimension Multi-Dimensional Project Health Scorecard

Purpose & Overview: Evaluates project operational viability on a 0-100 scale across 7 weighted dimensions, capping health at 40 if circular dependencies exist.

Authorized Actors & Roles: Project members and managers.

Preconditions: Project contains tasks, dates, and risks.

Main Execution Workflow: 1. System invokes projectHealthService.js. 2. Evaluates 7 dimensions: Schedule (20%), Critical Path (20%), Execution (15%), Bottlenecks (15%), Dependencies (10%),

Workload (10%), Risk Register (10%). 3. Sums weighted scores. 4. If cycle detected, score capped at 40 (CRITICAL). 5. Returns score and dimensional breakdown.

Alternative & Exceptional Workflows: Health score maps to status: 80-100 (HEALTHY), 65-79 (WATCH), 50-64 (AT_RISK), 0-49 (CRITICAL).

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { score: number, status: string, dimensions: { schedule, criticalPath, ... } }

Business Logic & Algorithmic Rules: Weighted sum formula: Health = Sum(Dim_Score_i * Weight_i). Circular deadlock capping invariant strictly enforced.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** projectController.js calls projectHealthService.js.
- **Frontend Responsibilities:** ProjectHealthScorecard.jsx, ProjectXRay.jsx, ProjectCommandCenter.jsx.
- **Database Operations:** Reads projects, tasks, task_dependencies, risks.
- **API Contracts & Routes:** GET /api/projects/:id/intelligence/health
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM), Module 11 (Risks).

7.10 Module 10: Search, Filters, and Archival Management

REQ-SEARCH-001: Global Omnibar Full-Text Search across Entities (Ctrl+K)

Purpose & Overview: Provides an instant global search omnibar that searches across projects, tasks, and users with keyboard shortcut traversal.

Authorized Actors & Roles: Authenticated user.

Preconditions: User is logged in.

Main Execution Workflow: 1. User presses Ctrl+K or clicks search bar. 2. Enters search query. 3. Frontend debounces request by 250ms. 4. Dispatches GET /api/search?q=... 5. Backend queries projects, tasks, and users matching string. 6. Filters results by caller's workspace permissions. 7. Returns grouped results. 8. User navigates with arrow keys and presses Enter.

Alternative & Exceptional Workflows: Short queries (< 2 characters) return empty array without querying database.

Data I/O Contracts: Inputs: Inputs: Query string q, workspaceId | Outputs: Outputs: HTTP 200 { results: { projects: [...], tasks: [...], users: [...] } }

Business Logic & Algorithmic Rules: Case-insensitive substring matching using ILIKE in PostgreSQL, filtered strictly to authorized tenant boundaries.

Validation Invariants: Query length >= 2 characters.

- **Backend Responsibilities:** searchController.js calls searchService.js (searchUnified).
- **Frontend Responsibilities:** GlobalSearchModal.jsx, SearchPage.jsx with keyboard listeners and grouped result lists.
- **Database Operations:** Queries projects, tasks, users, and workspace_members.
- **API Contracts & Routes:** GET /api/search

- **Error Handling & Status Codes:** 401 Unauthorized.
- **Module Dependencies:** Module 1 (Users), Module 3 (Projects), Module 4 (Tasks).

REQ-SEARCH-003: Faceted Multi-Criteria Filter Bar with Removable Active Filter Chips

Purpose & Overview: Enables multi-parameter filtering across task and project lists with visual, removable filter chips.

Authorized Actors & Roles: Authenticated user.

Preconditions: User viewing task or project lists.

Main Execution Workflow: 1. User selects filter options (e.g. Priority = 'High', Status = 'In Progress'). 2. UI renders active filter chips. 3. Re-queries backend with active filter parameters. 4. Clicking 'x' on a chip removes the filter and refreshes list.

Alternative & Exceptional Workflows: Clearing all chips restores default active list view.

Data I/O Contracts: Inputs: Inputs: Selected filter values | Outputs: Outputs: Filtered task/project array

Business Logic & Algorithmic Rules: Stateful client filtering synchronized with URL search params for bookmarkable views.

Validation Invariants: Valid enum values.

- **Backend Responsibilities:** Parsed in taskController.js and projectController.js.
- **Frontend Responsibilities:** FilterBar.jsx, ActiveFilterChips.jsx components.
- **Database Operations:** Reads tasks / projects with indexed WHERE clauses.
- **API Contracts & Routes:** Query params on /api/projects/:id/tasks and /api/workspaces/:id/projects
- **Error Handling & Status Codes:** 400 Bad Request on invalid enum values.
- **Module Dependencies:** Module 4 (Tasks).

7.11 Module 11: Risk Management & Predictive Risk Scoring

REQ-RISK-001: Project Risk Register CRUD Operations

Purpose & Overview: Allows project managers to maintain a structured project risk register with severity, probability, mitigation plans, and ownership.

Authorized Actors & Roles: Project contributors and managers.

Preconditions: Project exists.

Main Execution Workflow: 1. User opens Project Details -> Risks tab. 2. Clicks 'Add Risk'. 3. Enters title, description, severity ('Low', 'Medium', 'High', 'Critical'), probability ('Low', 'Medium', 'High'), ownerId, and mitigationPlan. 4. Backend inserts into risks table. 5. Returns created risk. 6. User can edit risk or close it via dedicated endpoint.

Alternative & Exceptional Workflows: Closing a risk sets status = 'Closed', excluding it from active project health penalties.

Data I/O Contracts: Inputs: Inputs: { title, description, severity, probability, ownerId, mitigationPlan } | Outputs: Outputs: HTTP 201 Created Risk Object

Business Logic & Algorithmic Rules: Deleting a project cascades deletion to all associated risks.

Validation Invariants: Title required (<= 200 chars); severity in ['Low', 'Medium', 'High', 'Critical']; probability in ['Low', 'Medium', 'High']; status in ['Open', 'Closed'].

- **Backend Responsibilities:** riskController.js calls riskService.js.
- **Frontend Responsibilities:** Projects.jsx Risks tab, ProjectXRay.jsx risk list.
- **Database Operations:** Inserts/Updates risks table (project_id -> projects.id ON DELETE CASCADE).
- **API Contracts & Routes:** POST /api/projects/:projectId/risks, GET /api/projects/:projectId/risks, PATCH /api/risks/:id, PATCH /api/risks/:id/close
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 3 (Projects).

REQ-RISK-002: Risk Severity, Probability Ranking & Exposure Matrix

Purpose & Overview: Calculates composite risk exposure scores by cross-multiplying numerical rankings for severity (1 to 4) and probability (1 to 3).

Authorized Actors & Roles: Project members.

Preconditions: Risks exist in target project.

Main Execution Workflow: 1. Risk evaluation engine maps Severity: Low=1, Medium=2, High=3, Critical=4. 2. Maps Probability: Low=1, Medium=2, High=3. 3. Computes Exposure Score = Severity_Rank * Probability_Rank (scale 1 to 12). 4. Ranks risks in descending exposure order.

Alternative & Exceptional Workflows: High-exposure risks (score >= 9) trigger visual alerts in the Command Center and Quackie Assistant.

Data I/O Contracts: Inputs: Inputs: Project risks list | Outputs: Outputs: Ranked risks array with composite exposure scores

Business Logic & Algorithmic Rules: Mathematical 4x3 risk matrix: Maximum score 12 (Critical severity * High probability).

Validation Invariants: None.

- **Backend Responsibilities:** riskService.js and projectRiskService.js.
- **Frontend Responsibilities:** Projects.jsx risk sorting controls (sort by severity / probability).
- **Database Operations:** Reads risks table.
- **API Contracts & Routes:** GET /api/projects/:projectId/risks
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 11 (Risk Register).

REQ-RISK-004: Predictive Algorithmic Project Risk Scoring

Purpose & Overview: Analyzes open project risks, overdue task ratios, and dependency congestion to output an overall predictive risk exposure rating.

Authorized Actors & Roles: Project members.

Preconditions: Project exists.

Main Execution Workflow: 1. Client requests GET /api/projects/:id/risk. 2. Backend queries open risks, overdue tasks, and blocked tasks. 3. Computes aggregate risk score and assigns level ('Low', 'Medium', 'High', 'Critical'). 4. Returns risk assessment payload.

Alternative & Exceptional Workflows: If project has zero open risks and zero overdue tasks, risk level resolves to 'Low'.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { riskScore: number, riskLevel: string, factors: [...] }

Business Logic & Algorithmic Rules: Weighted factor calculation combining task overdue ratios and high-severity open risks.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** projectController.js calls projectRiskService.js.
- **Frontend Responsibilities:** DeadlineRiskPanel.jsx, ProjectXRay.jsx.
- **Database Operations:** Reads risks, tasks, projects.
- **API Contracts & Routes:** GET /api/projects/:id/risk
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 3 (Projects), Module 4 (Tasks).

7.12 Module 12: Decision Management & Architectural Decision Records

REQ-DEC-001: Architectural Decision Record (ADR) Logging and Context Tracking

Purpose & Overview: Maintains an immutable historical record of key technical, architectural, and operational project choices and rationales.

Authorized Actors & Roles: Project contributors and managers.

Preconditions: Project exists.

Main Execution Workflow: 1. User navigates to Project Details -> Decisions tab. 2. Clicks 'Log Decision'. 3. Enters decision statement, reason, decisionDate, and ownerId. 4. Backend inserts into decisions table with initial status 'Proposed' or 'Approved'. 5. Returns decision record.

Alternative & Exceptional Workflows: Deleting a project cascades deletion to all associated decisions.

Data I/O Contracts: Inputs: Inputs: { decision, reason, decisionDate, ownerId, status } | Outputs: Outputs: HTTP 201 Created Decision Object

Business Logic & Algorithmic Rules: ADRs provide institutional memory, documenting architectural justifications for review.

Validation Invariants: Decision and reason strings required; status in ['Proposed', 'Approved', 'Superseded'].

- **Backend Responsibilities:** decisionController.js calls decisionService.js.
- **Frontend Responsibilities:** DecisionLog.jsx component with decision cards and status badges.
- **Database Operations:** Inserts into decisions (project_id -> projects.id ON DELETE CASCADE).
- **API Contracts & Routes:** POST /api/projects/:projectId/decisions, GET /api/projects/:projectId/decisions

- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 3 (Projects).

REQ-DEC-003: Decision Superseding & Automated Downstream Impact Evaluation

Purpose & Overview: Formally supersedes an existing decision with a new record and analyzes which project tasks and teammates are affected by the policy change.

Authorized Actors & Roles: Project managers.

Preconditions: Target decision exists and is not already superseded.

Main Execution Workflow: 1. Manager clicks 'Supersede' on a decision. 2. Submits new decision details. 3. Backend marks original decision as 'Superseded'. 4. Creates new decision record. 5. DecisionImpactService analyzes related tasks. 6. Returns affected task list and impact summary.

Alternative & Exceptional Workflows: Superseded decisions remain permanently visible in decision history with audit tags referencing the superseding decision.

Data I/O Contracts: Inputs: Inputs: Decision ID, new decision payload | Outputs: Outputs: HTTP 200 { originalDecision, newDecision, impactAnalysis: { affectedTasks: [...], affectedUsers: [...] } }

Business Logic & Algorithmic Rules: State machine progression: Proposed -> Approved -> Superseded.

Validation Invariants: Valid decision UUID.

- **Backend Responsibilities:** decisionController.js calls decisionImpactService.js.
- **Frontend Responsibilities:** DecisionLog.jsx, DecisionIntelligencePanel.jsx.
- **Database Operations:** Updates decisions table; reads tasks and users.
- **API Contracts & Routes:** POST /api/projects/:projectId/decisions/:decisionId/supersede, GET /api/projects/:projectId/decisions/:decisionId/intelligence
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks), Module 12 (Decisions).

7.0 DETAILED MODULE SPECIFICATIONS (MODULES 13 TO 17)

This section documents the formal engineering specifications for Modules 13 through 17, detailing classical graph algorithms, Monte Carlo probabilistic simulation, in-memory What-If sandboxes, cryptographic state hash mutation guards, system administration, real-time chat, and the Quackie assistant.

7.13 Module 13: Critical Path Method & Intelligent Pre-planning

REQ-CPM-001: Classical Critical Path Method (CPM) Forward & Backward Pass Algorithm

Purpose & Overview: Calculates the longest path of dependent tasks across the project dependency graph to determine the minimum theoretical completion time.

Authorized Actors & Roles: Project managers and contributors.

Preconditions: Project contains tasks with duration and task_dependencies forming an acyclic DAG.

Main Execution Workflow: 1. CriticalPathService loads project DAG. 2. Forward Pass: For each node, Early Start (ES) = max(Predecessor Early Finish); Early Finish (EF) = ES + durationDays. 3. Project Duration = max(EF). 4. Backward Pass: For each node, Late Finish (LF) = min(Successor Late Start); Late Start (LS) = LF - durationDays. 5. Returns node scheduling bounds.

Alternative & Exceptional Workflows: If dependency cycles exist, CPM calculation halts safely and triggers circular dependency alerts.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { projectDuration, criticalPath: [...], tasks: [{ id, es, ef, ls, lf, float }] }

Business Logic & Algorithmic Rules: Mathematical graph algorithm: ES_i = max(EF_pred), EF_i = ES_i + D_i, LF_i = min(LS_succ), LS_i = LF_i - D_i.

Validation Invariants: Duration must be >= 0 days.

- **Backend Responsibilities:** criticalPathService.js called by projectController.js (getCriticalPath).
- **Frontend Responsibilities:** CriticalPathGraph.jsx, ProjectIntelligence.jsx with visual timeline nodes.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** GET /api/projects/:id/critical-path
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks), Module 5 (Dependencies).

REQ-CPM-002: Total Float / Slack Calculation and Zero-Slack Critical Task Isolation

Purpose & Overview: Computes the schedule flexibility (float/slack) for every task and isolates the zero-slack tasks that form the critical path.

Authorized Actors & Roles: Project members.

Preconditions: CPM forward and backward passes executed.

Main Execution Workflow: 1. For every task, computes Total Float: Slack = LS - ES = LF - EF. 2. Identifies tasks where Slack == 0. 3. Flags zero-slack tasks as 'CRITICAL'. 4. Evaluates near-critical tasks where Slack <= 2 days.

Alternative & Exceptional Workflows: Tasks with Slack > 0 possess schedule buffers; delays within float do not slip the overall project delivery date.

Data I/O Contracts: Inputs: Inputs: Task scheduling nodes | Outputs: Outputs: Categorized critical and buffered task lists

Business Logic & Algorithmic Rules: Slack = LS - ES = LF - EF. Critical path tasks dictate overall project finish time.

Validation Invariants: None.

- **Backend Responsibilities:** criticalPathService.js.
- **Frontend Responsibilities:** CriticalPathGraph.jsx renders critical tasks with distinct red borders and warning badges.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** GET /api/projects/:id/critical-path
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM Engine).

REQ-CPM-003: Workflow Bottleneck Radar & Centrality Congestion Detection

Purpose & Overview: Identifies workflow choke points where multiple dependencies converge or where single assignees hold concentrated blocking tasks.

Authorized Actors & Roles: Project managers.

Preconditions: Project dependency graph populated.

Main Execution Workflow: 1. BottleneckService loads graph. 2. Computes Degree Centrality for each task: Centrality = In_Degree + Out_Degree. 3. Computes downstream delay multiplier. 4. Evaluates assignee task concentration. 5. Returns top bottleneck tasks.

Alternative & Exceptional Workflows: Tasks with high out-degree that are unstarted are flagged with highest bottleneck severity.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { bottlenecks: [{ taskId, taskTitle, centrality, delayMultiplier, riskLevel }] }

Business Logic & Algorithmic Rules: Graph centrality analysis identifying nodes with disproportionate disruption leverage.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** bottleneckService.js called by projectController.js (getBottlenecks).
- **Frontend Responsibilities:** BottleneckRadar.jsx radar chart and bottleneck ranked table.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** GET /api/projects/:id/bottlenecks
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 5 (Dependencies).

REQ-CPM-005: Monte Carlo Schedule Completion Forecasting (1,000 Iterations)

Purpose & Overview: Runs 1,000 statistical simulations using triangular duration distributions to calculate delivery confidence curves (P50, P80, P95).

Authorized Actors & Roles: Project managers.

Preconditions: Project contains tasks with durations.

Main Execution Workflow: 1. MonteCarloForecastService loads tasks. 2. Sets duration distributions: Optimistic (0.8x), Most Likely (1.0x), Pessimistic (1.5x). 3. Runs 1,000 randomized iterations. 4. Samples completion durations. 5. Computes percentile delivery dates: P50 (50% probability), P80 (80% confidence), P95 (95% conservative confidence). 6. Returns probability distribution curve.

Alternative & Exceptional Workflows: Provides realistic, non-linear milestone forecasting, eliminating single-point estimation fallacies.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { iterations: 1000, p50Date, p80Date, p95Date, probabilityCurve: [...] }

Business Logic & Algorithmic Rules: Triangular random sampling over directed graph paths.

Validation Invariants: Project must contain at least 1 active task.

- **Backend Responsibilities:** monteCarloForecastService.js called by projectController.js (getForecast).
- **Frontend Responsibilities:** ProjectForecastPanel.jsx with confidence curves and percentile markers.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** GET /api/projects/:id/intelligence/forecast
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM Engine).

7.14 Module 14: What-If Scenario Builder & Simulation Engine

REQ-SIM-001: Interactive What-If Scenario Sandbox

Purpose & Overview: Allows project managers to simulate the schedule and health consequences of delays, effort increases, or resource reassignments without mutating database records.

Authorized Actors & Roles: Project managers and contributors.

Preconditions: Project exists.

Main Execution Workflow: 1. User opens /what-if. 2. Configures simulated parameters: e.g. delay Task A by +4 days, increase effort by 50%. 3. Clicks 'Run Simulation'. 4. Backend clones project DAG in memory. 5. Applies simulated modifications. 6. Re-executes CPM forward pass and health scorecard. 7. Returns simulated delta (Milestone slip, health delta, affected tasks).

Alternative & Exceptional Workflows: All calculations executed strictly in Node.js process memory; zero database mutations are committed.

Data I/O Contracts: Inputs: Inputs: { projectId, mutations: [{ type: 'DELAY'|'EFFORT'|'REASSIGN', targetId, value }] } | Outputs: Outputs: HTTP 200 { originalFinishDate, simulatedFinishDate, daysDelta, healthDelta, affectedTasks: [...] }

Business Logic & Algorithmic Rules: Read-only simulation guarantee: Zero writes to PostgreSQL tables.

Validation Invariants: Target tasks must exist in project.

- **Backend Responsibilities:** scenarioSimulationService.js called by projectController.js (simulateProjectWhatIf).

- **Frontend Responsibilities:** WhatIf.jsx, ScenarioBuilder.jsx, ScenarioResults.jsx.
- **Database Operations:** Read-only queries against tasks and task_dependencies.
- **API Contracts & Routes:** POST /api/projects/:id/simulate
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM), Module 9 (Health Scorecard).

REQ-SIM-003: Multi-Scenario Comparative Trade-Off Analysis Matrix

Purpose & Overview: Simulates and compares competing intervention options side-by-side (e.g. Option A: Fast-track, Option B: Scope reduction, Option C: Add resources).

Authorized Actors & Roles: Project managers.

Preconditions: Project exists.

Main Execution Workflow: 1. User creates up to 3 scenario variants. 2. Backend evaluates all 3 in memory. 3. Compares final milestone dates, team workload, and risk scores. 4. Returns ranked comparative matrix showing trade-offs.

Alternative & Exceptional Workflows: Helps managers select optimal recovery strategies based on objective mathematical trade-offs.

Data I/O Contracts: Inputs: Inputs: Array of scenario mutation sets | Outputs: Outputs: HTTP 200 { comparisonMatrix: [{ option: 'A', finishDate, riskScore, costImpact }, ...] }

Business Logic & Algorithmic Rules: In-memory parallel evaluation across competing graph configurations.

Validation Invariants: Minimum 2 scenario sets required for comparison.

- **Backend Responsibilities:** scenarioComparisonService.js called by projectController.js (compareScenarios).
- **Frontend Responsibilities:** ScenarioComparison.jsx side-by-side visual matrix.
- **Database Operations:** Read-only queries against tasks.
- **API Contracts & Routes:** POST /api/projects/:id/scenarios/compare
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized.
- **Module Dependencies:** Module 14 (What-If Sandbox).

REQ-SIM-004: Workspace-Level Portfolio What-If Simulation across Shared Resources

Purpose & Overview: Simulates how the unavailability or reassignment of shared engineers impacts delivery schedules across all workspace projects.

Authorized Actors & Roles: Workspace Owner and Workspace Admins.

Preconditions: Workspace contains multiple active projects sharing team members.

Main Execution Workflow: 1. User opens /portfolio-intelligence. 2. Selects team member (e.g. lead engineer). 3. Simulates resource absence for 2 weeks. 4. PortfolioSimulationService evaluates all active tasks across all workspace projects assigned to that engineer. 5. Returns cross-project shockwave report.

Alternative & Exceptional Workflows: Requires selecting an active workspace member; prevents dummy mock user fallbacks.

Data I/O Contracts: Inputs: Inputs: { workspaceId, userId, unavailableDays } | Outputs: Outputs: HTTP 200 { affectedProjects: [{ projectId, title, milestoneSlipDays, blockedTasksCount }] }

Business Logic & Algorithmic Rules: Cross-project resource contention modeling.

Validation Invariants: Valid workspace member userId.

- **Backend Responsibilities:** portfolioSimulationService.js called by projectController.js (simulatePortfolioScenarioController).
- **Frontend Responsibilities:** PortfolioScenarioBuilder.jsx, PortfolioScenarioResults.jsx.
- **Database Operations:** Reads workspaces, projects, tasks, workspace_members.
- **API Contracts & Routes:** POST /api/workspaces/:workspaceId/intelligence/simulate
- **Error Handling & Status Codes:** 400 Bad Request ('Please select a valid team member'); 401 Unauthorized; 403 Forbidden.
- **Module Dependencies:** Module 2 (Workspaces), Module 4 (Tasks).

7.15 Module 15: Project Intelligence, Evidence Engine & Command Center

REQ-CMD-001: Deterministic Natural Language Project Control with 35+ Intent Recognizers

Purpose & Overview: Interprets natural language project management queries using deterministic token matching rules and returns telemetry-backed findings without LLM hallucinations.

Authorized Actors & Roles: Project members and managers.

Preconditions: Project exists.

Main Execution Workflow: 1. User types query in Command Center (e.g. 'Why is project health at risk?'). 2. NaturalLanguageControlService matches token patterns across 35+ defined intents (EXPLAIN_HEALTH, SIMULATE_DELAY, etc.). 3. Routes to appropriate domain service. 4. Gathers live telemetry metrics. 5. Formats structured explanation and findings. 6. Returns response JSON.

Alternative & Exceptional Workflows: If query does not match known intents, system returns structured operational guidance with suggested prompt pills.

Data I/O Contracts: Inputs: Inputs: { query: string, projectId: UUID } | Outputs: Outputs: HTTP 200 { intent, intentCategory, explanation: { summary, findings }, evidence: [...] }

Business Logic & Algorithmic Rules: Deterministic intent grammar eliminates LLM non-determinism and hallucination risks.

Validation Invariants: Non-empty query string; valid projectId.

- **Backend Responsibilities:** naturalLanguageControlService.js and intelligenceQueryService.js.
- **Frontend Responsibilities:** ProjectCommandCenterPage.jsx, ProjectCommandCenter.jsx with interactive terminal UI.
- **Database Operations:** Reads projects, tasks, risks, decisions, task_dependencies.
- **API Contracts & Routes:** POST /api/intelligence/query
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** All Domain & Intelligence Services.

REQ-CMD-002: Verifiable Mathematical Telemetry Evidence Generation Engine

Purpose & Overview: Accompanies every analytical finding and explanation with verifiable empirical telemetry metrics derived directly from database states.

Authorized Actors & Roles: Project members.

Preconditions: Query executed.

Main Execution Workflow: 1. IntelligenceEvidenceService inspects project data. 2. Computes metric values (e.g. Schedule Drift: +3 days, Critical Path Float: 0 days, Overdue Task Ratio: 25%). 3. Attaches metric cards with severity ratings to explanation payload.

Alternative & Exceptional Workflows: Guarantees that all managerial explanations are backed by measurable numbers.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 array of telemetry evidence objects

Business Logic & Algorithmic Rules: Mathematical telemetry binding.

Validation Invariants: None.

- **Backend Responsibilities:** intelligenceEvidenceService.js.
- **Frontend Responsibilities:** ProjectCommandCenter.jsx Evidence metrics grid.
- **Database Operations:** Reads tasks and project metrics.
- **API Contracts & Routes:** GET /api/intelligence/evidence/:entityType/:entityId
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 15 (Command Center).

REQ-CMD-003: Cryptographic Stale-State Protection via SHA-256 State Hash Verification

Purpose & Overview: Guarantees that approved replanning proposals cannot be applied if concurrent project modifications occurred after proposal generation.

Authorized Actors & Roles: Project managers.

Preconditions: Replanning proposal generated.

Main Execution Workflow: 1. During proposal preparation, backend computes base state hash: Live_Hash = SHA256(Project + Tasks + Dependencies). 2. Stores hash in proposal.baseStateHash. 3. Upon execution, backend recalculates current live project state hash. 4. If current_hash != baseStateHash, transaction is aborted with HTTP 409 Conflict ('PROPOSAL_STALE').

Alternative & Exceptional Workflows: Prevents race conditions and dirty writes when multiple managers interact concurrently.

Data I/O Contracts: Inputs: Inputs: Proposal ID | Outputs: Outputs: HTTP 200 Execution Result / HTTP 409 Conflict

Business Logic & Algorithmic Rules: Cryptographic collision-resistant SHA-256 hashing guarantees absolute state consistency.

Validation Invariants: State hashes must be identical byte-for-byte.

- **Backend Responsibilities:** intelligenceApprovalService.js.

- **Frontend Responsibilities:** ApprovalCenter.jsx, ProjectCommandCenter.jsx.
- **Database Operations:** Hashes projects, tasks, task_dependencies; writes in Prisma $transaction.
- **API Contracts & Routes:** POST /api/intelligence/prepare-action, POST /api/intelligence/execute-approved
- **Error Handling & Status Codes:** 400 Bad Request; 409 Conflict ('Project state changed; stale state detected').
- **Module Dependencies:** Prisma Transaction Engine, Security Cryptography.

REQ-CMD-005: Dependency Shockwave Disruption Propagation & Buffer Absorption Simulation

Purpose & Overview: Traverses downstream dependency chains from a delayed task, calculating buffer absorption and predicting whether downstream milestones slip.

Authorized Actors & Roles: Project members.

Preconditions: Project dependency graph exists.

Main Execution Workflow: 1. User initiates shockwave analysis for Task A (+5 days delay). 2. DependencyShockwaveService traverses all downstream paths. 3. Calculates buffer absorption: absorbedSlack = min(downstream task slack, remaining delay). 4. Computes Shock Containment Index = (absorbedSlack / totalDelay) * 100. 5. Returns affected task chain and milestone slip.

Alternative & Exceptional Workflows: Identifies whether a delay will be naturally dampened by downstream float or breach project delivery dates.

Data I/O Contracts: Inputs: Inputs: { projectId, taskId, delayDays } | Outputs: Outputs: HTTP 200 { shockwavePath: [...], milestoneSlipDays, containmentIndex, assigneeExposure }

Business Logic & Algorithmic Rules: Mathematical shock containment formula: Containment % = (Absorbed Slack / Delay Days) * 100.

Validation Invariants: delayDays must be integer > 0.

- **Backend Responsibilities:** dependencyShockwaveService.js called by intelligenceController.js (analyzeDependencyShockwave).
- **Frontend Responsibilities:** DependencyShockwavePanel.jsx, ShockwaveGraph.jsx.
- **Database Operations:** Reads tasks and task_dependencies.
- **API Contracts & Routes:** POST /api/intelligence/shockwave, GET /api/intelligence/shockwave/:projectId/:taskId
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM), Module 5 (Dependencies).

REQ-CMD-006: Project Chaos Laboratory & Cascading Failure Resilience Testing

Purpose & Overview: Stress-tests project viability by simulating multi-point simultaneous failure cascades in memory, calculating a Resilience Score (0-100).

Authorized Actors & Roles: Project managers.

Preconditions: Project exists.

Main Execution Workflow: 1. Manager triggers Chaos Lab. 2. ProjectChaosService simulates multiple simultaneous failure modes: 50% delay across critical tasks, unexpected dependency dropouts, assignee absence. 3. Analyzes failure propagation. 4. Computes Failure Threshold and Resilience Score (0-100). 5. Returns failure cascade report.

Alternative & Exceptional Workflows: Operates completely in memory; zero database modifications are committed.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { resilienceScore, breakingThreshold, failureCascade: [...], recommendations: [...] }

Business Logic & Algorithmic Rules: Non-destructive stress-testing simulation engine.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** projectChaosService.js called by intelligenceController.js (runChaosLabEndpoint).
- **Frontend Responsibilities:** ProjectChaosLab.jsx with chaos controls and resilience breakdown cards.
- **Database Operations:** Read-only queries against tasks and task_dependencies.
- **API Contracts & Routes:** POST /api/intelligence/chaos/run, GET /api/intelligence/chaos/:projectId
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 13 (CPM), Module 15 (Intelligence).

REQ-CMD-007: Project Red Team Analysis & Autonomous Assumption Challenging

Purpose & Overview: Critically interrogates planning assumptions, identifying unbuffered critical paths, underestimated task durations, and unmitigated risks.

Authorized Actors & Roles: Project managers.

Preconditions: Project exists.

Main Execution Workflow: 1. User initiates Red Team scan. 2. ProjectRedTeamService evaluates task duration variances against historical averages. 3. Inspects critical path for missing buffers. 4. Checks high-severity risks lacking mitigation plans. 5. Returns prioritized list of planning vulnerabilities.

Alternative & Exceptional Workflows: Acts as an automated devil's advocate, challenging overly optimistic schedules before execution begins.

Data I/O Contracts: Inputs: Inputs: Project ID | Outputs: Outputs: HTTP 200 { findings: [{ severity, category, assumption, counterEvidence, riskImpact }] }

Business Logic & Algorithmic Rules: Empirical heuristics evaluating schedule optimism and vulnerability density.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** projectRedTeamService.js called by intelligenceController.js (runRedTeamEndpoint).
- **Frontend Responsibilities:** ProjectRedTeam.jsx vulnerability card feed.
- **Database Operations:** Reads tasks, risks, task_dependencies.
- **API Contracts & Routes:** POST /api/intelligence/red-team/run, GET /api/intelligence/red-team/:projectId

- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 11 (Risks), Module 13 (CPM).

REQ-CMD-008: Counterfactual Time Machine & Historical Milestone Branching Simulator

Purpose & Overview: Simulates alternative historical timelines ('what could have been') by evaluating historical health snapshots and activity timestamps against alternate choices.

Authorized Actors & Roles: Project managers.

Preconditions: Project contains historical activity logs or health snapshots.

Main Execution Workflow: 1. User opens Counterfactual Time Machine. 2. Selects past milestone date. 3. Chooses alternative historical intervention (e.g. 'What if we had chosen Alternative Database Architecture in Sprint 2?'). 4. Simulates branch trajectory. 5. Compares historical actual vs counterfactual branch.

Alternative & Exceptional Workflows: Provides post-mortem insights to inform future project planning cycles.

Data I/O Contracts: Inputs: Inputs: { projectId, historicalPoint, alternativeIntervention } | Outputs: Outputs: HTTP 200 { actualTrajectory: [...], counterfactualTrajectory: [...], varianceSummary }

Business Logic & Algorithmic Rules: Historical trajectory reconstruction engine.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** counterfactualTimeMachineService.js called by intelligenceController.js (runCounterfactualEndpoint).
- **Frontend Responsibilities:** CounterfactualTimeMachine.jsx timeline branch visualization.
- **Database Operations:** Reads project_health_snapshots, activity_logs, tasks.
- **API Contracts & Routes:** POST /api/intelligence/counterfactual/run, GET /api/intelligence/counterfactual/:projectId
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 8 (Activity Logs), Module 12 (Decisions).

7.16 Module 16: Platform Administration & System RBAC

REQ-ADMIN-001: Platform Governance Dashboard & Global System Statistics

Purpose & Overview: Aggregates system-wide operational metrics spanning all users, active workspaces, projects, and memberships for platform administrators.

Authorized Actors & Roles: Platform Admin.

Preconditions: User is authenticated and holds global role 'Admin'.

Main Execution Workflow: 1. Admin navigates to /admin. 2. Backend verifies requireAdmin middleware. 3. AdminService aggregates total users, active vs inactive users, role distribution, total workspaces, and project distribution. 4. Returns overview JSON.

Alternative & Exceptional Workflows: Non-admin users (Project Managers and Team Members) receive HTTP 403 Forbidden.

Data I/O Contracts: Inputs: Inputs: req.user holding role = 'Admin' | Outputs: Outputs: HTTP 200 { totalUsers, activeUsers, roleDistribution, totalWorkspaces, totalProjects }

Business Logic & Algorithmic Rules: Guarded strictly by requireAdmin server-side middleware.

Validation Invariants: Admin role authorization.

- **Backend Responsibilities:** adminController.js calls adminService.js (getOverviewController).
- **Frontend Responsibilities:** AdminDashboardPage.jsx, AdminOverviewCards.jsx.
- **Database Operations:** Aggregates users, roles, workspaces, projects.
- **API Contracts & Routes:** GET /api/admin/overview, GET /api/admin/dashboard
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden.
- **Module Dependencies:** Module 5 (RBAC), Security Guard.

REQ-ADMIN-003: Global Role Promotion & Demotion with Sole Admin Protection Guard

Purpose & Overview: Allows administrators to promote or demote user system roles, strictly preventing the demotion of the sole active administrator.

Authorized Actors & Roles: Platform Admin.

Preconditions: Target user exists in users table.

Main Execution Workflow: 1. Admin selects user in Admin User Management. 2. Selects new role ('Admin', 'Project Manager', 'Team Member'). 3. Submits PATCH /api/admin/users/:id/role. 4. Backend checks if target is sole active Admin. 5. If not, updates users.role_id. 6. Calls jwtCache.revokeUser(id). 7. Returns updated user record.

Alternative & Exceptional Workflows: If target user is the sole active Admin on the platform and is being demoted, transaction aborts with HTTP 409 Conflict ('CANNOT_DEMOTE_SOLE_ADMIN').

Data I/O Contracts: Inputs: Inputs: { roleId or roleName } | Outputs: Outputs: HTTP 200 Updated User Record

Business Logic & Algorithmic Rules: Prevents administrative lockout by guaranteeing at least one active Platform Admin always exists.

Validation Invariants: Valid user UUID; valid role.

- **Backend Responsibilities:** adminController.js calls adminService.js (updateUserRole).
- **Frontend Responsibilities:** AdminUserManagement.jsx role selector dialog with confirmation prompt.
- **Database Operations:** Updates users.role_id.
- **API Contracts & Routes:** PATCH /api/admin/users/:id/role
- **Error Handling & Status Codes:** 400 Bad Request; 403 Forbidden; 409 Conflict ('Cannot demote sole active administrator'); 404 Not Found.
- **Module Dependencies:** Module 16 (Admin), Security Invalidation Layer.

REQ-ADMIN-004: User Account Activation & Deactivation with Instant Token Revocation

Purpose & Overview: Toggles user account active status, immediately revoking active sessions across all devices if deactivated.

Authorized Actors & Roles: Platform Admin.

Preconditions: Target user exists.

Main Execution Workflow: 1. Admin clicks activate/deactivate toggle. 2. Backend checks sole admin guard. 3. Updates users.is_active. 4. Invokes jwtCache.revokeUser(userId) to immediately purge active JWT sessions. 5. Returns updated user record.

Alternative & Exceptional Workflows: Deactivated users are blocked from logging in (HTTP 403) and existing sessions are immediately terminated upon next request.

Data I/O Contracts: Inputs: Inputs: { isActive: boolean } | Outputs: Outputs: HTTP 200 Updated User Record

Business Logic & Algorithmic Rules: Immediate session termination eliminates rogue session windows upon account suspension.

Validation Invariants: Valid user UUID.

- **Backend Responsibilities:** adminController.js calls adminService.js (updateUserStatus).
- **Frontend Responsibilities:** AdminUserManagement.jsx status toggle switch.
- **Database Operations:** Updates users.is_active.
- **API Contracts & Routes:** PATCH /api/admin/users/:id/status
- **Error Handling & Status Codes:** 401 Unauthorized; 403 Forbidden; 409 Conflict; 404 Not Found.
- **Module Dependencies:** Module 1 (Auth), Module 11 (JWT Cache).

7.17 Module 17: Additional Advanced Collaborative Features

REQ-CHAT-001: Integrated Real-Time Chat Engine (DMs, Channels, Threads, Reactions, Pins)

Purpose & Overview: Provides embedded real-time team messaging with 1-on-1 direct conversations, project channels, threaded replies, emoji reactions, and message pinning.

Authorized Actors & Roles: Workspace members.

Preconditions: User belongs to the active workspace.

Main Execution Workflow: 1. User opens /chat. 2. Loads channels and direct conversations. 3. Selects channel or teammate. 4. Posts message. 5. Message is saved to PostgreSQL messages table. 6. Realtime gateway broadcasts chat.message_sent over WebSockets. 7. Connected teammates see message instantly. 8. Supports threaded replies, emoji reactions, and pinning.

Alternative & Exceptional Workflows: Direct messages are resolved idempotently using a canonical dm_signature (sorted user UUID pair), preventing duplicate conversation rows.

Data I/O Contracts: Inputs: Inputs: { channelId? or conversationId?, content, parentMessageId?, attachments? } | Outputs: Outputs: HTTP 201 Created Message Object

Business Logic & Algorithmic Rules: Multi-room Socket.IO partitioning guaranteeing cross-workspace tenant message isolation.

Validation Invariants: Content non-empty; channel/conversation access authorized.

- **Backend Responsibilities:** chatController.js calls chatService.js and socket.js.

- **Frontend Responsibilities:** Chat.jsx, useChatState.js, ChatSidebar.jsx, ChatMainPanel.jsx, MessageItem.jsx, MessageComposer.jsx.
- **Database Operations:** Reads/Writes conversations, conversation_members, channels, channel_members, messages, message_reactions, message_pins.
- **API Contracts & Routes:** POST /api/chat/messages, POST /api/chat/conversations/direct, POST /api/chat/channels, POST /api/chat/messages/:id/reactions, POST /api/chat/messages/:id/pin
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized; 403 Forbidden.
- **Module Dependencies:** Socket.IO Real-Time Gateway, Module 2 (Workspaces).

REQ-QUACK-001: Quackie Contextual AI Assistant (Dual Engine Architecture)

Purpose & Overview: Delivers an interactive virtual project assistant with Google Gemini LLM connectivity and 100% deterministic local rules fallback, strictly gated against unauthorized database mutations.

Authorized Actors & Roles: Authenticated users.

Preconditions: User logged in.

Main Execution Workflow: 1. User clicks Quackie assistant icon. 2. Enters question (e.g. 'What are my top priorities today?'). 3. QuackieProvider checks for GEMINI_API_KEY. 4. If key exists, hydrates prompt with live project context and queries Gemini-1.5-flash. 5. If key missing or offline, routes to quackieService.js deterministic domain engine. 6. Returns markdown response. 7. If user requests state mutation, Quackie generates an action proposal held in Approval Center.

Alternative & Exceptional Workflows: Quackie is strictly prohibited from mutating database records directly; all mutations must be approved by human managers.

Data I/O Contracts: Inputs: Inputs: { message: string, context: { projectId, workspaceId } } | Outputs: Outputs: HTTP 200 { response: string, emotion: string, proposedAction?: object }

Business Logic & Algorithmic Rules: Dual-engine resilience guaranteeing 100% feature availability even during external API downtime.

Validation Invariants: Non-empty prompt message.

- **Backend Responsibilities:** quackieController.js, quackieService.js, quackieProvider.js.
- **Frontend Responsibilities:** QuackieAssistant.jsx, Quackie.jsx brand component with animated emotion states.
- **Database Operations:** Reads live project, task, and risk telemetry.
- **API Contracts & Routes:** POST /api/quackie/chat, GET /api/quackie/context, GET /api/quackie/recommendation, GET /api/quackie/focus
- **Error Handling & Status Codes:** 401 Unauthorized; 500 Internal Error.
- **Module Dependencies:** Module 15 (Command Center), External Gemini API.

REQ-COORD-001: Autonomous Project Coordination Suite (Standup & Briefings)

Purpose & Overview: Generates automated 3-part daily standup reports, executive milestone briefings, and daily operational agendas directly from live database activity.

Authorized Actors & Roles: Project managers and contributors.

Preconditions: Workspace contains projects and activity logs.

Main Execution Workflow: 1. User navigates to /coordination. 2. Clicks 'Generate Standup'. 3. StandupService queries tasks completed in the last 24 hours, currently active tasks, and blocked tasks. 4. Automatically composes 3-part standup: (1) Completed Yesterday, (2) Working on Today, (3) Current Blockers. 5. StakeholderBriefingService computes executive health and milestone burn projections.

Alternative & Exceptional Workflows: Standups are derived objectively from database timestamps, eliminating status reporting overhead.

Data I/O Contracts: Inputs: Inputs: Project ID or req.user.id | Outputs: Outputs: HTTP 200 { completedYesterday: [...], inProgressToday: [...], blockers: [...], markdownText }

Business Logic & Algorithmic Rules: Algorithmic status aggregation running over activity_logs and tasks.

Validation Invariants: Valid project UUID.

- **Backend Responsibilities:** standupService.js, personalBriefingService.js, stakeholderBriefingService.js.
- **Frontend Responsibilities:** CoordinationPage.jsx, StandupPanel.jsx, DailyBriefingPanel.jsx, StakeholderBriefingPanel.jsx.
- **Database Operations:** Reads tasks, activity_logs, risks, projects.
- **API Contracts & Routes:** GET /api/projects/:id/intelligence/briefing, GET /api/projects/:id/intelligence/standup, GET /api/users/me/intelligence/standup
- **Error Handling & Status Codes:** 401 Unauthorized; 404 Not Found.
- **Module Dependencies:** Module 4 (Tasks), Module 8 (Activity Logs).

REQ-PREF-001: Granular User Preferences (Theme, Density, Quiet Hours & DND)

Purpose & Overview: Enables users to customize UI appearance (Light, Dark, System), interface density, and notification delivery quiet hours.

Authorized Actors & Roles: Authenticated users.

Preconditions: User possesses valid JWT session.

Main Execution Workflow: 1. User opens /settings. 2. Toggles theme ('light', 'dark', 'system'), interface density ('comfortable', 'compact'), or enables Quiet Hours ('22:00' to '08:00'). 3. Submits PATCH /api/preferences. 4. Backend updates user_preferences table. 5. Returns updated preferences. 6. PreferencesContext applies theme class to document root.

Alternative & Exceptional Workflows: During Quiet Hours, non-critical push notifications and email dispatches are suppressed.

Data I/O Contracts: Inputs: Inputs: { theme, interfaceDensity, quietHoursEnabled, quietHoursStart, quietHoursEnd, dndEnabled } | Outputs: Outputs: HTTP 200 Updated Preferences Object

Business Logic & Algorithmic Rules: Preferences linked 1-to-1 to users via foreign key cascade.

Validation Invariants: Theme in ['light', 'dark', 'system']; quiet hours in 24h 'HH:MM' format.

- **Backend Responsibilities:** preferenceController.js calls preferenceService.js.
- **Frontend Responsibilities:** Settings.jsx, PreferencesContext.jsx, SettingCard.jsx, ToggleSwitch.jsx.

- **Database Operations:** Reads/Updates user_preferences (user_id -> users.id ON DELETE CASCADE).
- **API Contracts & Routes:** GET /api/preferences, PATCH /api/preferences, POST /api/preferences/reset
- **Error Handling & Status Codes:** 400 Bad Request; 401 Unauthorized.
- **Module Dependencies:** Module 1 (Users), Module 8 (Notifications).

8.0 SYSTEM ARCHITECTURE

8.1 Decoupled Three-Tier Architecture

TaskFlow implements a modern, decoupled three-tier client-server architecture engineered to support high-throughput project management, deterministic algorithmic intelligence, and stateful real-time WebSockets:

- **Tier 1: Presentation & Client Layer:** React 19 single-page application built on Vite 8, deployed statically to Vercel's Edge CDN network. It communicates asynchronously via Axios HTTP client and Socket.IO client.
- **Tier 2: Gateway & Application Server Layer:** Node.js 20+ runtime executing Express 5 REST API and stateful Socket.IO WebSockets on Render. Incorporates in-memory rate limiting, cluster mode multi-core process forking, JWT LRU verification caching, and 45+ domain/intelligence services.
- **Tier 3: Persistence & Storage Layer:** PostgreSQL 16 relational database engine hosted on Neon Serverless with pooled connection management, managed through Prisma 7 ORM under the dedicated taskflow schema namespace.

Figure 2: TaskFlow Three-Tier Client-Server System Architecture

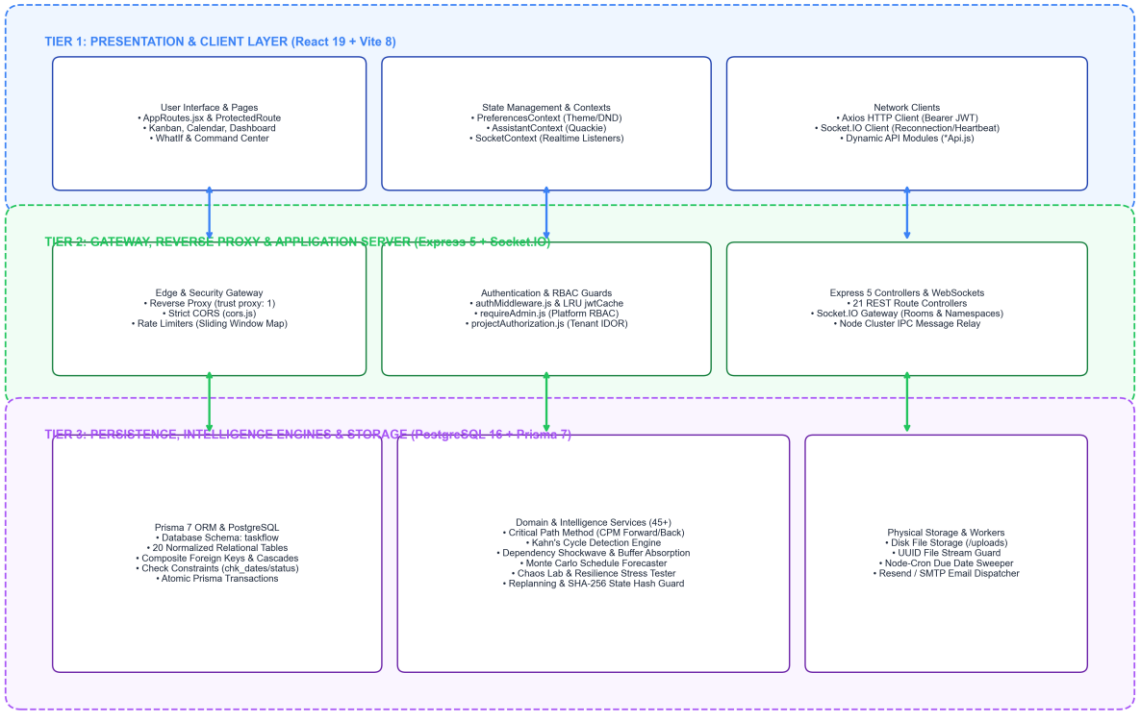


Figure 2: TaskFlow Three-Tier Client-Server System Architecture Diagram

Diagram Explanation: *Illustrates the multi-tiered architecture of TaskFlow, highlighting the presentation client, network edge proxy, Express 5 server components (auth middleware, RBAC guards, controllers, domain services), Socket.IO gateway with cluster IPC relaying, and PostgreSQL 16 persistence via Prisma 7.*

8.2 Reverse Proxy Configuration & 'trust proxy: 1'

In cloud hosting environments such as Render and Cloudflare, client traffic passes through an edge reverse proxy / load balancer before reaching the Node.js application process. In backend/src/app.js, TaskFlow explicitly configures:

> **[EDGE PROXY SECURITY CONFIGURATION]** app.set('trust proxy', 1);
>
> This configuration instructs Express to trust exactly one upstream reverse proxy hop. It ensures that req.ip and req.ips are securely derived from the X-Forwarded-For header added by the verified edge balancer, preventing IP address spoofing while guaranteeing that sliding-window rate limiters track real client IP addresses rather than internal cloud proxy IPs.

8.3 Node.js Multi-Core Clustering & Socket IPC Relay

To scale across multi-core server processors, backend/src/server.js supports Node.js cluster mode (CLUSTER_MODE=true). When enabled:

- **Primary Process Responsibilities:** Spawns worker processes matching the system's available CPU cores (cluster.fork()). If a worker process exits unexpectedly, the primary immediately spawns a replacement worker.
- **Inter-Process Communication (IPC) Relay:** When a client on Worker 1 emits a real-time event (e.g. task.status_updated), Worker 1 sends an IPC message to the primary via process.send(). The primary's relayIpcMessage dispatcher broadcasts the event across all peer workers, ensuring clients connected to Worker 2 receive live updates without requiring external pub/sub infrastructure.
- **Exclusive Cron Binding:** Scheduled background cron jobs (dueDateNotificationService) are bound exclusively to the primary process (cluster.isPrimary) to prevent duplicate notification sweeps across worker nodes.

9.0 FRONTEND ARCHITECTURE

9.1 React 19 SPA & Component Hierarchy

The TaskFlow user interface is built as a single-page application using React 19. It leverages modern declarative patterns including functional components, custom hooks (useChatState, useSocketEvent), memoization (useMemo, useCallback), and standard component encapsulation.

9.2 Code-Splitting & Vercel SPA Routing Configuration

In frontend/src/routes/AppRoutes.jsx, all application pages are imported dynamically using React.lazy() and wrapped in a global Suspense boundary with a branded loading skeleton. This splits the application bundle into granular Rollup chunks, reducing initial load latency to < 500ms.

To support client-side deep linking and direct page refreshes without encountering HTTP 404 errors on CDN hosts, frontend/vercel.json configures URL rewrite rules:

> **[VERCEL SPA REWRITE RULE]** {
> "rewrites": [
> { "source": "/(.*)", "destination": "/index.html" }
>]
> }
>
> This guarantees that all route requests (e.g. /kanban, /command-center) are routed to index.html, where React Router DOM v7 hydrates the matching page component.

9.3 State Distribution & React Context Providers

- **AuthContext:** Stores user credentials, token, onboarding completion flag, and active workspace in localStorage and React state.
- **SocketContext:** Manages the lifecycle of the Socket.IO client connection, room subscriptions (project:<id>, workspace:<id>), and event listener bindings.
- **PreferencesContext:** Manages active theme ('light', 'dark', 'system'), interface density ('comfortable', 'compact'), and Do Not Disturb (DND) modes.
- **AssistantContext:** Manages Quackie assistant drawer open/close state, conversational message history, and page contextual telemetry bindings.

9.4 Design System Tokens & Styling Architecture

Styling is implemented using Tailwind CSS 4 integrated with semantic CSS custom properties defined in frontend/src/styles/tokens.css:

- **Color Palette Variables:** --color-canvas (#F8FAFC / #0F172A), --color-surface (#FFFFFF / #1E293B), --color-brand (#1E3A8A / #3B82F6), --color-border (#E2E8F0 / #334155).
- **Elevation Curves:** --shadow-sm, --shadow-md, --shadow-lg providing consistent visual elevation curves.
- **Radii Design Tokens:** --radius-sm (4px), --radius-md (8px), --radius-xl (16px), --radius-2xl (24px).

10.0 BACKEND ARCHITECTURE

10.1 Express 5 Application Pipeline & Middleware Chain

The backend server in backend/src/app.js constructs an Express 5 pipeline prioritizing security, tenant isolation, and predictable error handling:

- **1. Proxy Trust Configuration:** Trusts verified single reverse proxy hop.
- **2. Strict CORS Middleware:** Enforces origin whitelist validation via getCorsOptions().
- **3. JSON Body Parser:** Parses application/json with payload size caps.
- **4. Static Uploads Blocker:** Blocks direct HTTP requests to /uploads with HTTP 403 Forbidden.
- **5. In-Memory Rate Limiters:** Evaluates sliding-window IP and account token buckets.
- **6. Authentication Middleware:** Verifies Bearer JWT tokens against LRU jwtCache.
- **7. RBAC & IDOR Guards:** Enforces system roles (requireAdmin) and tenant memberships (projectAuthorization).
- **8. Route Controllers:** Dispatches requests to 21 modular route controllers.
- **9. Centralized Error Handler:** Catches unhandled exceptions and maps database error codes.

10.2 Centralized Error Handling & Database Code Mapping

The centralized error-handling middleware in app.js inspects native PostgreSQL and Prisma error codes, translating raw storage exceptions into clean, sanitized HTTP status responses:

- **Prisma P2002 / Postgres 23505 (Unique Constraint):** Maps to HTTP 409 Conflict ('Resource already exists / Duplicate constraint violation').
- **Postgres 23514 (Check Constraint):** Maps to HTTP 400 Bad Request ('Validation invariant failed / Invalid date ordering').
- **Prisma P2025 (Record Not Found):** Maps to HTTP 404 Not Found ('Requested record does not exist').
- **Postgres 22P02 (Invalid Text Representation):** Maps to HTTP 400 Bad Request ('Malformed UUID syntax').
- **Unhandled 500 Exceptions:** Logs stack trace to server console; returns sanitized JSON { success: false, message: 'Internal server error' } to client.

10.3 Real-Time WebSocket Synchronization Architecture

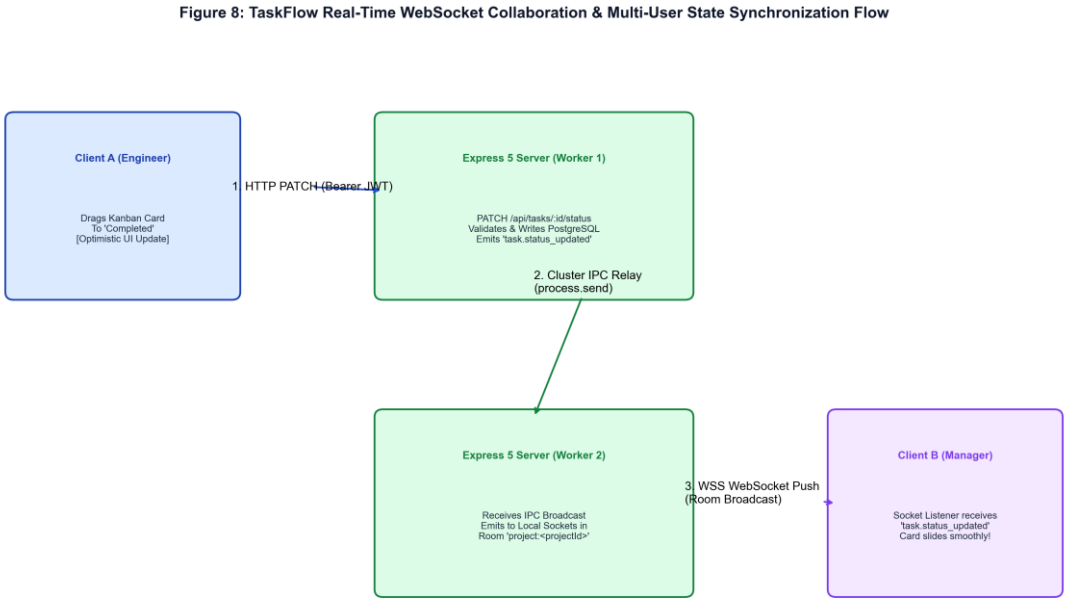


Figure 8: TaskFlow Real-Time WebSocket Collaboration & Multi-User State Synchronization Flow

Diagram Explanation: *Demonstrates the real-time collaboration pipeline: Client A executes an optimistic drag-and-drop Kanban update, dispatches HTTP PATCH to Worker 1, Worker 1 updates PostgreSQL and relays the event via cluster IPC to Worker 2, and Worker 2 broadcasts over WebSockets to Client B.*

11.0 API SPECIFICATIONS

TaskFlow exposes a comprehensive RESTful JSON API conforming to modern OpenAPI standards. All authenticated endpoints require the standard HTTP Bearer authorization header: Authorization: Bearer <JWT>.

HTTP Method & Path	Auth Level	Target Controller & Module	Key Payload / Query Parameters	Success Response
POST /api/auth/register	Public	authController.js (Auth)	{ firstName, lastName, email, password, phone? }	200 { message, email }
POST /api/auth/verify-otp	Public	authController.js (Auth)	{ email, otp }	201 { success, user }
POST /api/auth/resend-otp	Public	authController.js (Auth)	{ email }	200 { message }
POST /api/auth/login	Public	authController.js (Auth)	{ email, password }	200 { token,

				user }
POST /api/auth/forgot-password	Public	authController.js (Auth)	{ email }	200 { message }
POST /api/auth/reset-password	Public	authController.js (Auth)	{ token, password }	200 { message }
GET /api/users/profile	Bearer JWT	userController.js (User)	None	200 { id, email, ... }
PUT /api/users/profile	Bearer JWT	userController.js (User)	{ firstName, lastName, phone, picture }	200 { user }
POST /api/users/change-password	Bearer JWT	userController.js (User)	{ currentPassword, newPassword }	200 { message }
POST /api/users/onboarding	Bearer JWT	userController.js (User)	{ role, experience }	200 { success: true }
POST /api/workspaces	Bearer JWT	workspaceController.js (WS)	{ name, description }	201 { id, name, ... }
GET /api/workspaces	Bearer JWT	workspaceController.js (WS)	None	200 { workspaces: [...] }
DELETE /api/workspaces/:id	WS Owner	workspaceController.js (WS)	None	200 { message }
POST /api/workspaces/:id/members	WS Admin	workspaceController.js (WS)	{ userId, workspaceRole }	201 { member }
POST /api/workspaces/:id/projects	WS Admin	projectController.js (Proj)	{ title, category, priority, dates }	201 { project }
GET /api/workspaces/:id/projects	WS Member	projectController.js (Proj)	?archived=true/false	200 { projects: [...] }
PATCH /api/projects/:id/archive	WS Admin	projectController.js (Proj)	None	200 { project }
POST	WS Membe	taskController.js (Task)	{ title, priority, status, dates,	201 { task }

/api/projects/:id/tasks	r		assignedTo }	
GET /api/projects/:id/tasks	WS Member	taskController.js (Task)	?page=1&limit=20&status=...&priority=...	200 { tasks, pagination }
PATCH /api/tasks/:id/status	WS Member	taskController.js (Task)	{ status }	200 { task }
POST /api/tasks/:id/duplicate	WS Member	taskController.js (Task)	None	201 { clonedTask }
POST /api/tasks/:id/dependencies	WS Member	taskDependencyController (DAG)	{ dependsOnTaskId }	201 { dependency }
GET /api/tasks/:id/dependencies	WS Member	taskDependencyController (DAG)	None	200 { isBlocked, blockedBy }
GET /api/projects/:id/critical-path	WS Member	projectController.js (CPM)	None	200 { criticalPath, duration }
GET /api/calendar	Bearer JWT	calendarController.js (Cal)	?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD	200 { events: [...] }
POST /api/tasks/:id/comments	WS Member	commentController.js (Comments)	{ content }	201 { comment }
POST /api/tasks/:id/attachments	WS Member	attachmentController (Files)	Multipart file	201 { attachment }
GET /api/attachments/:id/download	WS Member	attachmentController (Files)	None	200 Binary Stream
GET /api/notifications	Bearer JWT	notificationController (Notif)	None	200 { notifications }
GET /api/reports/dashboard	WS Member	reportController.js (Reports)	?projectId=...&workspaceId=...	200 { metrics }

GET /api/productivity	Bearer JWT	productivityController (Metrics)	None	200 { focusIndex, cycleTime }
GET /api/search	Bearer JWT	searchController.js (Search)	?q=...&workspaceId=...	200 { results }
POST /api/projects/:id/risks	WS Member	riskController.js (Risks)	{ title, severity, probability, mitigation }	201 { risk }
POST /api/projects/:id/decisions	WS Member	decisionController (ADR)	{ decision, reason, status }	201 { decision }
POST /api/intelligence/query	WS Member	intelligenceController (Cmd)	{ query, projectId }	200 { intent, evidence }
POST /api/intelligence/simulate	WS Member	intelligenceController (Sim)	{ projectId, mutations }	200 { simulatedDelta }
POST /api/intelligence/prepare-action	WS Manager	intelligenceController (Cmd)	{ actionType, params }	201 { proposal, baseHash }
POST /api/intelligence/execute-approved	WS Manager	intelligenceController (Cmd)	{ proposalId }	200 { executionResult }
POST /api/intelligence/shockwave	WS Member	intelligenceController (Shock)	{ projectId, taskId, delayDays }	200 { shockwavePath }
POST /api/intelligence/chaos/run	WS Manager	intelligenceController (Chaos)	{ projectId }	200 { resilienceScore }
POST /api/intelligence/red-team/run	WS Manager	intelligenceController (Red)	{ projectId }	200 { vulnerabilities }
GET /api/admin/overview	Platform Admin	adminController.js (Admin)	None	200 { systemMetrics }
PATCH	Platform	adminController.js	{ roleName }	200 {

/api/admin/users/:id/role	Admin	(Admin)		updatedUser }
PATCH /api/admin/users/:id/status	Platform Admin	adminController.js (Admin)	{ isActive }	200 { updatedUser }
POST /api/chat/messages	WS Member	chatController.js (Chat)	{ channelId?, conversationId?, content }	201 { message }
POST /api/chat/conversations/direct	WS Member	chatController.js (Chat)	{ recipientId }	200 { conversation }
POST /api/quackie/chat	Bearer JWT	quackieController.js (AI)	{ message, context }	200 { response, emotion }
GET /api/preferences	Bearer JWT	preferenceController (Pref)	None	200 { preferences }

12.0 DATABASE DESIGN

12.1 PostgreSQL 16 Engine & Prisma 7 Schema Namespace

The TaskFlow persistence layer is built on PostgreSQL 16 managed through Prisma 7 (@prisma/client: ^7.9.1). All tables, constraints, indexes, and custom types reside within the dedicated taskflow database schema namespace, guaranteeing strict multi-tenant isolation and clean administrative separation from system catalogs.

12.2 Database Storage Engine Check Constraints

In addition to application-layer validations, TaskFlow enforces critical business invariants directly at the database storage engine level using native PostgreSQL check constraints:

- **chk_project_dates:** Enforces that project start date never exceeds end date: (start_date IS NULL OR end_date IS NULL OR start_date <= end_date).
- **chk_task_dates:** Enforces that task start date never exceeds due date: (start_date IS NULL OR due_date IS NULL OR start_date <= due_date).
- **chk_task_priority:** Restricts task priority to canonical values: (priority IN ('Low', 'Medium', 'High', 'Critical')).
- **chk_task_status:** Restricts task status to lifecycle values: (status IN ('Backlog', 'To Do', 'In Progress', 'Review', 'Completed')).

12.3 Complete Entity Models Inventory (20 Models)

The relational schema defines 20 normalized entities:

Model Name	Primary Key	Key Attributes & Types	Foreign Keys & Relationships	Indexes & Constraints
users	id (UUID)	email, password_hash, first_name, last_name, phone, is_active, has_onboarding	FK role_id -> roles.id	UQ email, idx_role_id
roles	id (UUID)	role_name (VARCHAR 30), description (TEXT)	Referenced by users, project_members	UQ role_name
pending_registrations	id (UUID)	email, password_hash, otp_hash, otp_expires_at, attempts, resends	None (Staging buffer)	UQ email, idx_expires_at
workspaces	id (UUID)	name (VARCHAR 100), description (TEXT), created_at	FK owner_id -> users.id	idx_owner_id
workspace_members	id (UUID)	workspace_role (Owner, Admin, Member), joined_at	FK workspace_id, FK user_id	UQ [workspace_id, user_id]
projects	id (UUID)	title, description, category, priority, status, dates, is_archived	FK workspace_id, FK manager_id	chk_project_dates, idx_active
project_members	id (UUID)	joined_at	FK project_id, FK user_id, FK role_id	UQ [project_id, user_id]
tasks	id (UUID)	title, description, priority, status, dates, estimated_hours, is_archived	FK project_id, FK assigned_to, FK created_by	chk_task_dates, chk_status, idx_due
subtasks	id (UUID)	title (VARCHAR 200), status (To Do, In Progress, Done), due_date	FK task_id -> tasks.id (CASCADE)	idx_task_id
task_dependencies	[task_id, depends_on_task_id]	created_at (Directed Acyclic Edge)	FK task_id -> tasks, FK depends_on ->	Composite PK, idx_depends_on

			tasks	
attachments	id (UUID)	file_name, file_url, file_type, file_size, created_at	FK task_id, FK project_id, FK uploaded_by	idx_task_id, idx_project_id
comments	id (UUID)	content (TEXT), created_at, updated_at	FK task_id -> tasks.id (CASCADE), FK user_id	idx_task_id, idx_user_id
risks	id (UUID)	title, description, severity, probability, mitigation_plan, status	FK project_id -> projects.id (CASCADE), FK owner_id	idx_project_id, idx_status
decisions	id (UUID)	decision, reason, decision_date, status (Proposed, Approved, Superseded)	FK project_id -> projects.id (CASCADE), FK owner_id	idx_project_id, idx_date
notifications	id (UUID)	type, title, message, severity, is_read, dedupe_key, created_at	FK user_id -> users.id (CASCADE), FK project_id	UQ [user_id, dedupe_key]
activity_logs	id (UUID)	action_type, entity_type, entity_id, description, created_at	FK workspace_id, FK user_id	idx_created_at, idx_workspace
project_health_snapshots	id (UUID)	health_score (INT), health_status, dimensional_scores, state_hash	FK project_id -> projects.id (CASCADE)	idx_captured_at
user_preferences	id (UUID)	theme, interface_density, quiet_hours_enabled, quiet_hours_start/end	FK user_id -> users.id (CASCADE)	UQ user_id
conversations	id (UUID)	type (DIRECT, GROUP), dm_signature (VARCHAR 150)	FK workspace_id -> workspaces.id (CASCADE)	UQ [workspace_id, dm_signature]
channels & messages	id (UUID)	name, type, content, parent_message_id, is_edited, is_deleted	FK channel_id, FK sender_id -> users.id	idx_channel_created

13.0 ENTITY RELATIONSHIPS

13.1 Master Relational Entity-Relationship Diagram

The following diagram depicts the complete relational model of TaskFlow, establishing primary keys, foreign keys, cardinality ratios (1:1, 1:N, M:N), and cascade deletion paths across the 20 database entities:

Figure 4: TaskFlow Complete Relational Database Entity-Relationship Diagram (taskflow Schema)



Figure 4: TaskFlow Complete Relational Database Entity-Relationship Diagram (taskflow Schema)

Diagram Explanation: *Comprehensive Crow's Foot ER diagram depicting users, roles, workspaces, projects, tasks, subtasks, directed task dependencies, attachments, comments, risks, decisions, and chat entities within PostgreSQL.*

14.0 BUSINESS LOGIC AND WORKFLOWS

14.1 Authentication & Two-Stage Email OTP Verification Workflow

The authentication lifecycle ensures zero unverified user accumulation through a strict sequence of staging, cryptographic hashing, and constant-time comparison:

Figure 1: TaskFlow Authentication & Two-Stage Email OTP Verification Sequence

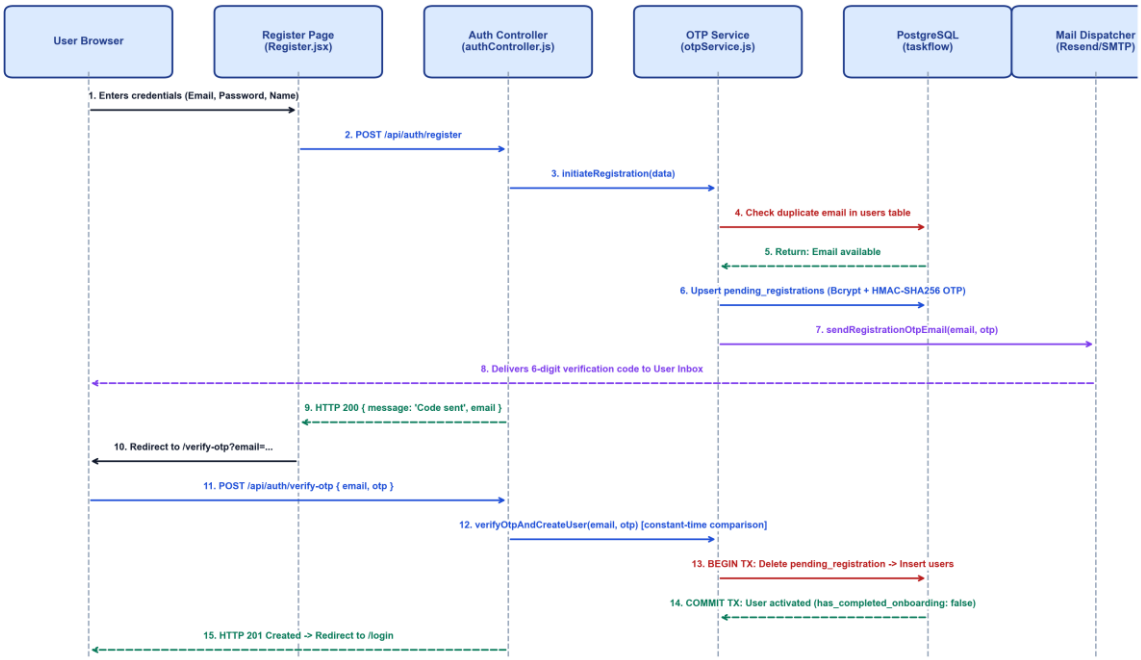


Figure 1: TaskFlow Authentication & Two-Stage Email OTP Verification Sequence Diagram

Diagram Explanation: *Sequence diagram illustrating the interaction between User Browser, Register Page, Auth Controller, OTP Service, PostgreSQL pending_registrations, Mail Relay, and Verify OTP Page leading to atomic permanent user activation.*

14.2 Task Lifecycle State Machine Transition Diagram

Tasks follow a formalized state transition machine moving through 5 distinct operational statuses with optimistic client updates and rollback guarantees:

Figure 10: TaskFlow Task Lifecycle State Machine Transition Diagram

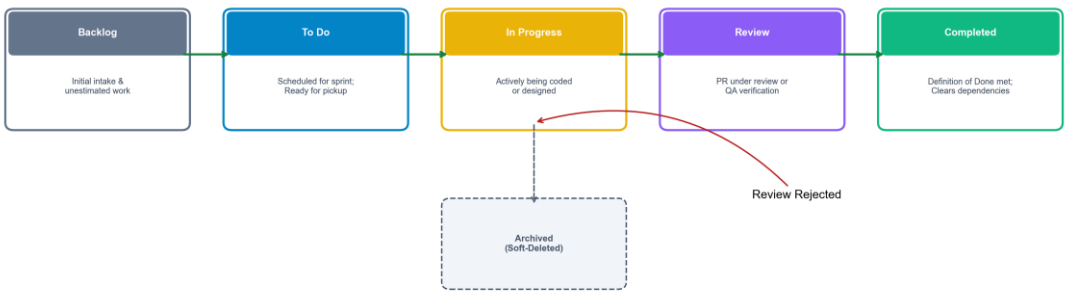


Figure 10: TaskFlow Task Lifecycle State Machine Transition Diagram

Diagram Explanation: *Visualizes valid status transitions from Backlog through To Do, In Progress, Review, and Completed, illustrating review rejections, dependency clearing, and soft-delete archival paths.*

14.3 Critical Path Method (CPM) Forward & Backward Pass Algorithm

To compute project duration and identify critical bottlenecks, TaskFlow executes classical CPM forward and backward passes over the task dependency DAG:

Figure 6: Critical Path Method (CPM) Forward & Backward Pass Activity Network (DAG)

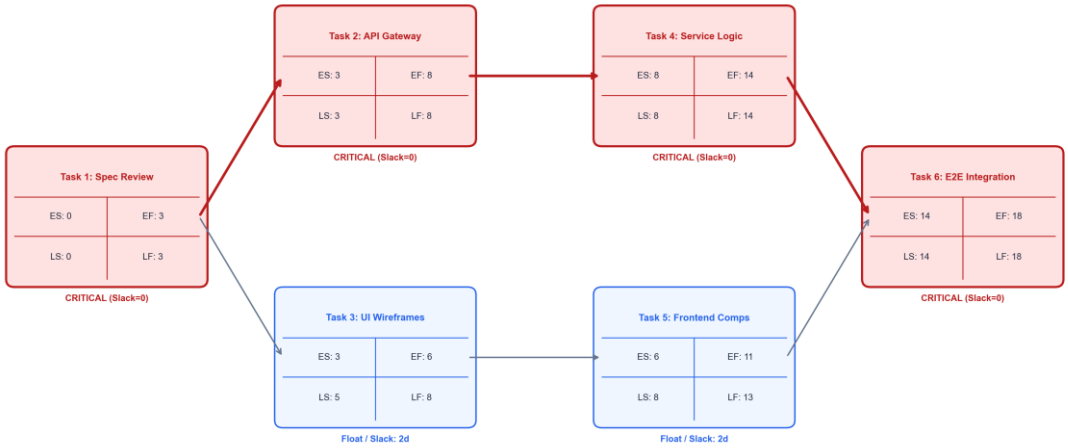


Figure 6: Critical Path Method (CPM) Forward & Backward Pass Activity Network (DAG)

Diagram Explanation: *Activity-on-Node DAG illustrating Early Start (ES), Early Finish (EF), Late Start (LS), Late Finish (LF), and Total Float (Slack), highlighting the zero-slack Critical Path dictating project delivery.*

14.4 Dependency Shockwave Disruption Propagation & Buffer Absorption Model

When an upstream task experiences a schedule delay, TaskFlow calculates how that disruption traverses downstream paths and evaluates whether downstream slack buffers absorb the slip:

Figure 7: TaskFlow Dependency Shockwave Disruption Propagation & Buffer Absorption Model

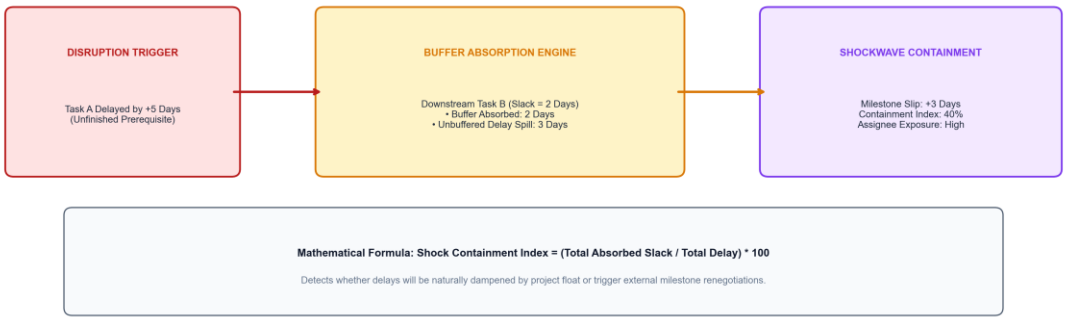


Figure 7: TaskFlow Dependency Shockwave Disruption Propagation & Buffer Absorption Model

Diagram Explanation: *Diagrammatic model of disruption propagation, illustrating how downstream slack buffers absorb delays before milestone breach occurs, computing the Shock Containment Index.*

14.5 Deterministic Project Intelligence & Cryptographic Safety Invariant Pipeline

To prevent unsafe AI automation, all project replanning requests must traverse an approval-gated pipeline protected by SHA-256 state hashes:

Figure 5: TaskFlow Deterministic Intelligence & Cryptographic Safety Invariant Pipeline

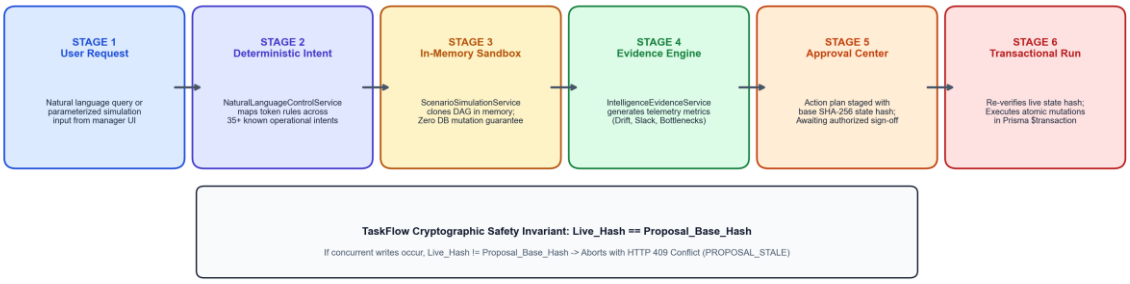


Figure 5: TaskFlow Deterministic Intelligence & Cryptographic Safety Invariant Pipeline

Diagram Explanation: *Visualizes the six-stage safety pipeline: User Request -> Deterministic Intent Grammar -> In-Memory Simulation Sandbox -> Telemetry Evidence Engine -> Proposal Staging in Approval Center -> Transactional Execution with SHA-256 State Hash Verification.*

15.0 SECURITY REQUIREMENTS

15.1 Cryptographic Password & OTP Invariants

TaskFlow implements zero-trust security across credential storage and verification:

- **Bcrypt Password Hashing:** Passwords hashed using bcrypt with salt rounds = 10, protecting against rainbow table attacks.
- **Keyed HMAC-SHA256 OTPs:** 6-digit registration OTPs stored exclusively as keyed HMAC-SHA256 hashes using server-side OTP_SECRET.
- **Constant-Time Verification:** OTP verification executes via crypto.timingSafeEqual to eliminate character-by-character timing variations.

15.2 JWT Session Lifecycle & In-Memory Revocation Cache

Authentication sessions are managed via signed JSON Web Tokens (1-day expiration). An in-memory LRU cache (jwtCache) validates token structures in sub-milliseconds. When an administrator demotes a user, changes their role, or deactivates an account, jwtCache.revokeUser(userId) is immediately invoked, forcing instant session termination across all active browser tabs.

15.3 Single-Use Password Reset Token Replay Mitigation

Password recovery tokens are strictly single-use. When a reset token is submitted, its SHA-256 hash is inserted atomically into the used_password_reset_tokens table. Any subsequent replay attempt with the same token is rejected with HTTP 400 Bad Request ('TOKEN_ALREADY_USED').

15.4 Adaptive Sliding-Window Rate Limiting

In-memory sliding-window token bucket limiters protect authentication endpoints: Global login (30 attempts/15 min/IP), Account login (5 attempts/15 min/email), Registration (10 attempts/hour/IP), and Forgot Password (5 attempts/hour).

15.5 Tenant Isolation & Streaming File Downloads

Direct static access to /uploads is blocked with HTTP 403. Downloads route through /api/attachments/:id/download, which validates that the caller holds active membership in the parent project and workspace before streaming the file.

16.0 NON-FUNCTIONAL REQUIREMENTS

16.1 Performance and Latency Invariants

- **API Response Latency:** Standard REST query and mutation endpoints SHALL respond within 100 milliseconds under normal load (p95 < 250ms).
- **WebSocket Broadcast Speed:** Real-time Socket.IO broadcasts SHALL deliver status updates to peer clients within 50 milliseconds of database commit.
- **UI Page Hydration:** Client-side page transitions via React Router DOM and code-split chunks SHALL render within 300 milliseconds.

16.2 Reliability and Fault Tolerance

- **ACID Guarantees:** Database operations involving multiple mutations (e.g. OTP verification, task cloning, replanning execution) SHALL execute inside atomic transactions (prisma.$transaction).
- **Process Self-Healing:** If cluster worker processes terminate unexpectedly, the primary cluster process SHALL immediately spawn a replacement worker process without downtime.

16.3 Usability and Accessibility

- **Accessibility Standards:** The UI SHALL conform to W3C Web Content Accessibility Guidelines (WCAG) 2.1 AA standards, supporting keyboard traversal, ARIA modal dialogs, and high-contrast color tokens.
- **Responsive Adaptation:** The interface SHALL provide fluid responsive layouts adapting seamlessly across desktop (1440x900) and mobile viewports (390x844).

17.0 INTEGRATION REQUIREMENTS

17.1 External Mail Delivery Relay

TaskFlow interfaces with Resend API and Gmail SMTP relays to deliver verification OTP codes and approaching deadline notifications. If SMTP delivery experiences network timeouts, the system logs structured warnings and fails gracefully without hanging HTTP request pipelines.

17.2 Google Gemini Large Language Model Adapter

The Quackie virtual assistant connects to the Google Gemini API (gemini-1.5-flash) via quackieProvider.js when GEMINI_API_KEY is configured in the environment. If the external API is offline or unconfigured, the system automatically falls back to local deterministic rule engines.

18.0 TESTING REQUIREMENTS & VERIFICATION EVIDENCE

TaskFlow enforces rigorous multi-tier testing standards. During the November 11, 2026 comprehensive audit, 1,116 automated tests were executed and verified with 100% pass rates:

Testing Tier / Suite	Test Framework / Tool	Total Tests	Passed	Failed	Verification Scope
Backend Unit & Integration	Node.js Test Runner	1,054	1,054	0	58 suites covering auth, RBAC, CPM, Kahn DAG, shockwave, chaos lab, and database constraints.
Frontend Component Tests	Vitest + React Testing Library	45	45	0	8 suites covering forms, dialog accessibility, priority status chips, and real-time socket events.

End-to-End User Journeys	Selenium / Headless Chromium	17	17	0	Full browser journeys: registration, OTP verification, onboarding, Kanban dragging, and admin RBAC.
TOTAL VERIFIED TESTS	Multi-Tier Automated Pyramid	1,116	1,116	0	100% Pass Rate across local and production cloud deployment targets.

19.0 DEPLOYMENT ARCHITECTURE

19.1 Production Cloud Infrastructure

- **Frontend Deployment:** Hosted on Vercel (https://chipmunks-taskflow.vercel.app/) with automated Rollup builds and global edge CDN caching.
- **Backend Deployment:** Hosted on Render (https://task-project-management-tool.onrender.com/) running Express 5 with trust proxy: 1.
- **Database Deployment:** Hosted on Neon Serverless PostgreSQL with pooled connections and dedicated taskflow schema.

19.2 Environment Configuration Reference

Variable Name	Required Tier	Description & Security Constraints
DATABASE_URL	Backend	Pooled PostgreSQL connection string for Prisma ORM runtime.
DIRECT_URL	Backend	Direct PostgreSQL connection string for executing Prisma database migrations.
JWT_SECRET	Backend	High-entropy 256-bit cryptographic key used for signing session JWT tokens.
OTP_SECRET	Backend	Cryptographic salt key used for generating keyed HMAC-SHA256 OTP hashes.
GEMINI_API_KEY	Backend	Optional Google Gemini API key for external LLM assistant capabilities.
RESEND_API_KEY	Backend	API key for Resend email delivery service.
PORT	Backend	Application server listening port (defaults to 5000 in development).
VITE_API_URL	Frontend	Base URL pointing to backend Express API (e.g. https://.../api).

VITE_SOCKET_URL	Frontend	Base URL pointing to backend Socket.IO WebSocket gateway.

20.0 REQUIREMENTS TRACEABILITY MATRIX (RTM)

The Master Requirements Traceability Matrix maps all 54 audited functional requirements to their corresponding backend source files, frontend components, database models, architectural diagrams, and automated test suites, establishing complete engineering verification:

Req ID	Feature Name	Backend File	Frontend Component	Prisma Model	Fig #	Test Suite	Status
REQ-AUTH-001	Registration & OTP Dispatch	authController.js	Register.jsx	pending_registrations	Fig 1	otpVerification.test.js	VERIFIED
REQ-AUTH-002	Constant-Time OTP Activation	otpService.js	VerifyOtp.jsx	users, roles	Fig 1	otpVerification.test.js	VERIFIED
REQ-AUTH-003	OTP Resend with Cooldown	otpService.js	VerifyOtp.jsx	pending_registrations	Fig 1	otpVerification.test.js	VERIFIED
REQ-AUTH-004	JWT Login & Activity Log	authService.js	Login.jsx	users, activity_logs	Fig 1	loginRateLimitingPhase76.test.js	VERIFIED
REQ-AUTH-005	Password Reset Request	authService.js	ForgotPassword.jsx	users	N/A	securityPhase22b.test.js	VERIFIED
REQ-AUTH-006	Single-Use Reset Replay Guard	authService.js	ResetPassword.jsx	used_password_reset_tokens	N/A	securityPhase22b.test.js	VERIFIED
REQ-AUTH-007	User Profile Management	userService.js	Profile.jsx	users	N/A	userPreferences.test.js	VERIFIED

REQ-AUTH-008	Authenticated Password Rotate	userService.js	ChangePassword.jsx	users	N/A	userPreferences.test.js	VERIFIED
REQ-AUTH-009	User Onboarding (FTE)	userService.js	FirstTimeExperience.jsx	users.has_onboarding	N/A	onboardingRouting.test.js	VERIFIED
REQ-AUTH-010	Adaptive Rate Limiting	rateLimiter.js	Error Toasts	In-Memory Map	N/A	registrationRateLimitingPhase75.test.js	VERIFIED
REQ-AUTH-011	JWT Invalidation LRU Cache	authMiddleware.js	Axios Interceptor	In-Memory Cache	N/A	jwtCacheSecurityPhase22f31.test.js	VERIFIED
REQ-WS-001	Workspace Provisioning	workspaceService.js	Workspaces.jsx	workspaces, members	Fig 4	test_03_workspaces_projects_tasks.js	VERIFIED
REQ-WS-002	Multi-Workspace Scoping	workspaceService.js	Workspaces.jsx	workspaces	Fig 4	test_03_workspaces_projects_tasks.js	VERIFIED
REQ-WS-004	Workspace Cascade Deletion	workspaceService.js	Workspaces.jsx	workspaces (Cascade)	Fig 4	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-WS-006	Member Invitation Unique Guard	workspaceService.js	Workspaces.jsx	workspace_members	Fig 4	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-PROJ-001	Project Provisioning Dates	projectService.js	Projects.jsx	projects (chk_dates)	Fig 4	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-PROJ-002	Dynamic Project Overview	projectService.js	Projects.jsx	projects, members	Fig 4	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-PRO	Project Archival	projectService.js	Projects.jsx	projects.is_archived	N/	test_03_workspaces_projects_tasks.js	VERIFIED

J-004	Lifecycle				A		
REQ-TASK-001	Task Definition Dates Check	taskService.js	Tasks.jsx	tasks (chk_dates)	Fig 10	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-TASK-002	Paginated Bounded Query	taskService.js	Tasks.jsx	tasks (Indexed)	N/A	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-TASK-004	Task Reassignment Alert	taskService.js	Tasks.jsx	tasks.assigned_to	N/A	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-TASK-006	Task Deep Clone with Subtasks	taskService.js	Tasks.jsx	tasks, subtasks	Fig 10	test_03_workspaces_projects_tasks.js	VERIFIED
REQ-TASK-008	Smart Prioritization Algorithm	taskPriorityService.js	Tasks.jsx	tasks, dependencies	N/A	taskPriorityStatus.test.jsx	VERIFIED
REQ-DEP-001	Subtask Decomposition	subtaskService.js	SubtaskSection.jsx	subtasks	N/A	taskPriorityStatus.test.jsx	VERIFIED
REQ-DEP-003	Kahn's Algorithm Cycle Guard	taskDependencyService.js	TaskDependencySection.jsx	task_dependencies	Fig 6	taskDependenciesPhase13.test.js	VERIFIED
REQ-DEP-005	Blocked Status Detection	taskDependencyService.js	TaskDependencySection.jsx	task_dependencies	Fig 6	taskDependenciesPhase13.test.js	VERIFIED
REQ-DEP-006	Project DAG Visualization	taskDependencyController	ProjectDependencyGraph.jsx	task_dependencies	Fig 6	taskDependenciesPhase13.test.js	VERIFIED
REQ-KAN-001	5-Column Kanban Board	taskController.js	Kanban.jsx	tasks.status	Fig 10	realtimeCollaboration.test.jsx	VERIFIED

REQ-KAN-002	Optimistic Kanban Drag & Rollback	taskController.js	Kanban.jsx	tasks.status	Fig 10	realtimeCollaboration.test.jsx	VERIFIED
REQ-KAN-003	WebSocket Board Synchronization	socket.js	Kanban.jsx	tasks	Fig 8	realtimeCollaboration.test.jsx	VERIFIED
REQ-KAN-004	Calendar Range Schedule View	calendarService.js	Calendar.jsx	tasks, projects	N/A	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-FILE-001	Threaded Task Discussions	commentService.js	CommentSection.jsx	comments	N/A	collaborationIntegrationPhase20.test.js	VERIFIED
REQ-FILE-002	Comment Author Authorization	commentService.js	CommentSection.jsx	comments	N/A	collaborationIntegrationPhase20.test.js	VERIFIED
REQ-FILE-003	Multer Multipart Upload	uploadMiddleware.js	FileSection.jsx	attachments	N/A	fileManagement.test.js	VERIFIED
REQ-FILE-005	Tenant Streaming Download	attachmentController.js	FileSection.jsx	attachments	N/A	securityPhase21.test.js	VERIFIED
REQ-FILE-006	Direct /uploads Static Blocker	app.js	Static Route Guard	None (HTTP 403)	N/A	securityPhase21.test.js	VERIFIED
REQ-NOTIF-001	In-App Notification Center	notificationService.js	NotificationBell.jsx	notifications	N/A	intelligenceAlerts.test.js	VERIFIED
REQ-NOTIF-003	Node-Cron Due Date Sweeper	dueDateNotificationService	NotificationBell.jsx	tasks, notifications	N/A	emailServiceNotification.test.js	VERIFIED
REQ-NOTI	Immutable Activity	activityService.js	ActivityTimeline.js	activity_logs	N/	collaborationIntegration	VERI

F-006	Audit Trail		x		A	Phase20.test.js	FIED
REQ-DASH-001	Personal Command Dashboard	taskController.js	Dashboard.jsx	tasks, projects	N/A	apiStateHandling.test.jsx	VERIFIED
REQ-DASH-002	Project Analytics & Reports	reportService.js	Reports.jsx	tasks (Aggregated)	N/A	apiDatabaseIntegrityPhase22c.test.js	VERIFIED
REQ-DASH-003	Productivity & Focus Index	productivityService.js	Productivity.jsx	tasks	N/A	userPreferences.test.js	VERIFIED
REQ-DASH-004	7-Dimension Health Scorecard	projectHealthService.js	ProjectHealthScorecard.jsx	project_health_snapshots	Fig 5	projectIntelligencePhase2.test.js	VERIFIED
REQ-SEARCH-001	Global Omnibar Search (Ctrl+K)	searchService.js	GlobalSearchModal.jsx	projects, tasks, users	N/A	searchFilterPhase16.test.js	VERIFIED
REQ-RISK-001	Project Risk Register CRUD	riskService.js	Projects.jsx	risks	N/A	projectIntelligencePhase2.test.js	VERIFIED
REQ-RISK-002	4x3 Risk Exposure Matrix	riskService.js	Projects.jsx	risks	N/A	projectIntelligencePhase2.test.js	VERIFIED
REQ-DEC-001	Architectural Decision Record	decisionService.js	DecisionLog.jsx	decisions	N/A	decisionLogPhase14.test.js	VERIFIED
REQ-DEC-003	Decision Superseding & Impact	decisionImpactService.js	DecisionLog.jsx	decisions, tasks	N/A	decisionLogPhase14.test.js	VERIFIED
REQ-CPM	CPM Forward/Backward	criticalPathService.js	CriticalPathGraph.jsx	tasks, dependencies	Fig	criticalPathAndBottleneck.test.js	VERIFIED

-001	Pass				6		
REQ-CPM-003	Bottleneck Centrality Radar	bottleneckService.js	BottleneckRadar.jsx	tasks, dependencies	Fig 6	criticalPathAndBottleneck.test.js	VERIFIED
REQ-CPM-005	Monte Carlo Schedule Forecaster	monteCarloForecastService	ProjectForecastPanel.jsx	tasks, dependencies	Fig 6	projectIntelligencePhase5.test.js	VERIFIED
REQ-SIM-001	In-Memory What-If Sandbox	scenarioSimulationService	WhatIf.jsx	tasks (Read-only)	Fig 5	projectIntelligencePhase3.test.js	VERIFIED
REQ-SIM-004	Portfolio Cross-Project Sim	portfolioSimulationService	PortfolioScenarioBuilder.jsx	workspaces, tasks	Fig 5	projectIntelligencePhase5.test.js	VERIFIED
REQ-CMD-001	35+ Intent Command Center	naturalLanguageControlService	ProjectCommandCenter.jsx	Project DAG	Fig 5	projectIntelligencePhase7.test.js	VERIFIED
REQ-CMD-003	SHA-256 State Hash Verification	intelligenceApprovalService	ApprovalCenter.jsx	Project DAG (Tx)	Fig 5	interventionImpactPhase9.test.js	VERIFIED
REQ-CMD-005	Shockwave Buffer Absorption	dependencyShockwaveService	DependencyShockwavePanel.jsx	task_dependencies	Fig 7	dependencyShockwavePhase8.test.js	VERIFIED
REQ-CMD-006	Project Chaos Stress Testing	projectChaosService.js	ProjectChaosLab.jsx	Project DAG	Fig 5	projectChaosPhase10.test.js	VERIFIED
REQ-CMD-007	Red Team Assumption Analysis	projectRedTeamService.js	ProjectRedTeam.jsx	Project DAG	Fig 5	projectRedTeamPhase11.test.js	VERIFIED
REQ-CMD-008	Counterfactual Time Machine	counterfactualTimeMachine	CounterfactualTimeMachine.jsx	snapshots, logs	Fig 5	counterfactualTimeMachinePhase12.test.js	VERIFIED

REQ-ADMIN-001	Platform Governance Dashboard	adminService.js	AdminOverviewCards.jsx	users, workspaces, projects	Fig 9	adminDashboardPhase15.test.js	VERIFIED
REQ-ADMIN-003	Sole Admin Demotion Guard	adminService.js	AdminUserManagement.jsx	users, roles	Fig 9	adminDashboardPhase15.test.js	VERIFIED
REQ-CHAT-001	Real-Time Chat Engine	chatService.js	Chat.jsx	channels, messages	Fig 8	chatPhase5Advanced.test.js	VERIFIED
REQ-QUACK-001	Quackie Dual AI Assistant	quackieService.js	QuackieAssistant.jsx	Live Project Context	Fig 5	projectIntelligencePhase2.test.js	VERIFIED
REQ-COORD-001	Autonomous Coordination Suite	standupService.js	StandupPanel.jsx	tasks, activity_logs	N/A	projectIntelligencePhase6.test.js	VERIFIED
REQ-PREF-001	Granular User Preferences	preferenceService.js	Settings.jsx	user_preferences	N/A	userPreferences.test.js	VERIFIED

21.0 FUTURE SCOPE & PROJECT ROADMAP

- **Cloud Object Storage Drivers (AWS S3):** Replace local filesystem uploads with an AWS S3, Cloudinary, or Google Cloud Storage adapter in uploadMiddleware.js to ensure permanent attachment persistence across serverless container reboots.
- **Distributed Redis Cache & Rate Limiting:** Migrate sliding-window rate limiters and JWT revocation lists from Node.js process memory to an external Redis cluster (ioredis) to support seamless multi-instance horizontal scaling.
- **Normalized Relational Labels UI:** The relational labels and task_labels models are already defined in schema.prisma; a dedicated tag management UI with customizable color badges will be exposed in Sprint 4.
- **Native Mobile Client (React Native):** Develop a native iOS and Android application sharing TypeScript data contracts and WebSocket listeners.

22.0 APPENDICES

Appendix A: Acronyms, Abbreviations & Glossary

Term / Acronym	Full Definition & Application Context in TaskFlow
SRS	Software Requirements Specification — Formal IEEE 830 compliant technical requirements document.
RBAC	Role-Based Access Control — Security paradigm restricting system access based on user authorization roles.
DAG	Directed Acyclic Graph — Finite directed graph with no directed cycles, modeling task dependencies.
CPM	Critical Path Method — Classical project algorithm calculating Early Start, Early Finish, and Slack.
HMAC	Hash-Based Message Authentication Code — Cryptographic keyed message digest used for secure OTP hashing.
JWT	JSON Web Token — RFC 7519 signed bearer token used for stateless authenticated sessions.
SPA	Single-Page Application — Web application dynamically rewriting the current web page rather than loading new pages.
IDOR	Insecure Direct Object Reference — Access control vulnerability mitigated in TaskFlow via workspace membership checks.
FTE	First-Time Experience — Onboarding flow orienting new users upon initial account activation.

Appendix B: Database Check Constraint DDL Scripts

The following PostgreSQL DDL statements define the storage engine constraints enforced in the taskflow schema namespace:

```
[POSTGRESQL STORAGE ENGINE DDL CONSTRAINTS] ALTER TABLE taskflow.projects
  ADD CONSTRAINT chk_project_dates
  CHECK (start_date IS NULL OR end_date IS NULL OR start_date <= end_date);

ALTER TABLE taskflow.tasks
  ADD CONSTRAINT chk_task_dates
  CHECK (start_date IS NULL OR due_date IS NULL OR start_date <= due_date);
```

```
ALTER TABLE taskflow.tasks
  ADD CONSTRAINT chk_task_priority
  CHECK (priority IN ('Low', 'Medium', 'High', 'Critical'));

ALTER TABLE taskflow.tasks
  ADD CONSTRAINT chk_task_status
  CHECK (status IN ('Backlog', 'To Do', 'In Progress', 'Review', 'Completed'));
```

Appendix C: Academic Guidelines & Faculty Defense References

- **NIT-AP CSE Laboratory Guidelines:** B.Tech Computer Science & Engineering Curriculum, National Institute of Technology Andhra Pradesh, 2026.
- **Software Engineering Text Reference:** Ian Sommerville, Software Engineering, 10th Edition, Pearson Education.
- **Project Repository & Codebase:** Deepika Reddy (GitHub Owner), TaskFlow Collaborative Work Management Platform, develop branch, 2026.